

Exercise 2

Deadline 18.11.2025

Anna Bütfering

Lea Neumann

Juan Provencio

Two dimensional data with bottleneck

Use pytorch to implement and train an autoencoder. The autoencoder's constructor should have four arguments:

input_size: the data dimension (so that we can later reuse the autoencoder in task 3)

bottleneck_size: the code dimension (also for reuse in task 3 in the present task the code dimension is always 1)

hidden_size: the dimension of the hidden layers (should be larger than the data dimension!)

layers: the number of hidden layers

Packages & visualisation functions

```
In [1]: # Torch
import torch
from torch import nn, optim
from torch import sqrt, exp
# Sklearn
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
# Etc.
import matplotlib.pyplot as plt
from tqdm import tqdm
## Plot options
from cycler import cycler
from matplotlib import rcParams
rcParams['axes.grid'] = True
rcParams['grid.linestyle'] = '--'
rcParams['grid.alpha'] = 0.5
rcParams['axes.labelsize'] = 20
rcParams['axes.prop_cycle'] = cycler('color', ['#C61A27', '#3891A6', \
                                             '#F79D65', '#FDE74C', \
                                             '#9BC53D', '#5BC0EB',
```

```

                                                    '#E55934', '#FA7921',
                                                    '#8D230F', '#1B998B'])
colors = rcParams['axes.prop_cycle'].by_key()['color']
rcParams['axes.titlesize'] = 22
rcParams['figure.figsize'] = (12, 7)
rcParams['figure.titlesize'] = 26
rcParams['font.size'] = 16
rcParams['image.cmap'] = 'magma'
rcParams['lines.markeredgewidth'] = 2
rcParams['lines.markerfacecolor'] = 'white'
rcParams['markers.fillstyle'] = 'none'
# Define textbox style for plots
textbox_style = dict(boxstyle='round', facecolor='white', alpha=0.8, edgecol

```

```

In [5]: ''' Plotting functions for Ex01 '''
# Function for visualising the results in 1.1:
def plot1_1(model, results, title, label, label_index):
    fig, axes = plt.subplots(1, 2)
    fig.suptitle(title, fontsize=26)

    # Plot MSE loss for different parameters
    for res in results:
        params, train_losses, _ = res
        axes[0].plot(train_losses, label=f"{label}: {params[label_index]}")
    axes[0].set_xlabel("Iterations")
    axes[0].set_ylabel("MSE Loss")
    axes[0].legend()
    axes[0].loglog()

    # Plot final reconstruction
    # Get final output
    final_epoch, X_batch, decoded = model.outputs[-1]
    axes[1].scatter(X_batch[:,0].detach().numpy(), X_batch[:,1].detach().numpy(),
                    label="Original Data",
                    color = colors[5])
    for i in range(len(results)):
        final_epoch, X_batch, decoded = results[i][2]
        axes[1].scatter(decoded[:,0].detach().numpy(), decoded[:,1].detach().numpy(),
                        label=f"({label}: {results[i][0][label_index]})",
                        color = colors[i])
    axes[1].set_xlabel("X1")
    axes[1].set_ylabel("X2")
    axes[1].legend()

    plt.tight_layout()
    plt.show()

# Function for visualising the results in 1.2:
def plot1_2(model, results, title):

    #Print out average and variance of MSE:
    all_avg = []
    print(f"Set: Mean, Std")
    for res in results:
        TS, _, train_losses, _ = res
        avg = np.mean(train_losses)

```

```

        all_avg.append(avg)
        std = np.std(train_losses)
        print(f"{TS}: {avg:.4f} {std:.4f}")
    total_mean = np.mean(all_avg)
    total_std = np.std(all_avg)
    print(f"Average MSE over all data sets: {total_mean:.4f} with a standard

fig, axes = plt.subplots(1, 3)
fig.suptitle(title, fontsize=26)

# Plot MSE loss for different training sets:
for res in results:
    TS, params, train_losses, _ = res
    axes[1].plot(train_losses, label=f"Training set: {TS}")
axes[1].set_xlabel("Iterations")
axes[1].set_ylabel("MSE Loss")
#axes[1].legend(loc='upper center', bbox_to_anchor=(0.5, -0.15), ncol=
axes[1].loglog()

# Plot final reconstruction
# Get final output
final_epoch, X_batch, decoded = model.outputs[-1]
for i in range(len(results)):
    final_epoch, X_batch, decoded = results[i][3]
    axes[0].scatter(decoded[:,0].detach().numpy(), decoded[:,1].detach()
                    label=f"(Training set: {results[i][0]})",
                    color = colors[i])
axes[0].set_xlabel("X1")
axes[0].set_ylabel("X2")
axes[0].legend(loc='upper center', bbox_to_anchor=(0.5, -0.15))

# Plot Boxplots of MSE:
data = []
labels = []
for res in results:
    TS, _, train_losses, __ = res
    data.append(np.array(train_losses).flatten())
    labels.append(TS)
axes[2].boxplot(data, tick_labels= labels, medianprops=dict(color='black
axes[2].set_yscale('log')
axes[2].set_xlabel(None)
axes[2].set_ylabel("MSE loss")

plt.tight_layout()
plt.show()

```

Autoencoder Class Implementation

```

In [2]: # Dataset from two moons:
X, y = make_moons(n_samples=125000, noise=0.1, random_state=42)
X = torch.tensor(X, dtype=torch.float32)

```

```
#y = torch.tensor(y, dtype=torch.long)

# Split into train and test sets:
X_test, X_train = train_test_split(X, test_size=0.2, random_state=42)
```

```
In [3]: class Autoencoder(nn.Module):
    def __init__(self, input_size, bottleneck_size, hidden_size, layers):
        # Initialize AE with required parameters
        self.input_size = input_size
        self.bottleneck_size = bottleneck_size
        self.hidden_size = hidden_size
        self.layers = layers
        # Save hyperparameters
        self.lr = 0.001
        # Save results
        self.train_losses = []
        self.test_losses = []
        self.outputs = []

        # Structure: [ENCODER -> {BOTTLENECK} -> DECODER]

        # First call super constructor
        super(Autoencoder, self).__init__()

        # Encoder
        ## Initial layer input -> hidden
        self.encoder = nn.ModuleList()
        self.encoder.append(nn.Linear(input_size, hidden_size))
        self.encoder.append(nn.ReLU())
        ## Hidden layers hidden -> hidden
        for _ in range(layers - 1):
            self.encoder.append(nn.Linear(hidden_size, hidden_size))
            self.encoder.append(nn.ReLU())
        ## Final layer hidden -> bottleneck
        self.encoder.append(nn.Linear(hidden_size, bottleneck_size))

        # Decoder
        ## Initial layer bottleneck -> hidden
        self.decoder = nn.ModuleList()
        self.decoder.append(nn.Linear(bottleneck_size, hidden_size))
        self.decoder.append(nn.ReLU())
        ## Hidden layers hidden -> hidden
        for _ in range(layers - 1):
            self.decoder.append(nn.Linear(hidden_size, hidden_size))
            self.decoder.append(nn.ReLU())
        ## Final layer hidden -> output
        self.decoder.append(nn.Linear(hidden_size, input_size))

    def forward(self, x):
        # Forward pass
        for layer in self.encoder:
            x = layer(x)
        for layer in self.decoder:
            x = layer(x)
        decoded = x
```

```

    return decoded

def train_autoencoder(self, X_train, training_set_size, mini_batch_size,
    # Loss function
    loss_function = nn.MSELoss()
    # Optimizer
    optimizer = optim.Adam(self.parameters(), lr = learning_rate)
    # Use Data loader for mini-batches, use training_set_size to select s
    loader = torch.utils.data.DataLoader(X_train[:training_set_size], ba

    for epoch in tqdm(range(n_epochs)):
        for X_batch in loader:
            #print(X_batch)
            # Make forward pass
            decoded = self.forward(X_batch)
            # Compute loss
            loss = loss_function(decoded, X_batch)
            # Compute zero-grad
            optimizer.zero_grad()
            # Backprop
            loss.backward()
            # Step
            optimizer.step()
            # Save losses
            self.train_losses.append(loss.item())
            # Save outputs for later
            self.outputs.append((epoch, X_batch, decoded))

#Functions for 1.3: Training of a GMM on the encoded data (Z)
'''
Only difference in forwardZ() and train_autoencoderZ() is that the encod
'''
def forwardZ(self, x):
    # Forward pass
    for layer in self.encoder:
        x = layer(x)
    encoded = x
    for layer in self.decoder:
        x = layer(x)
    decoded = x
    return decoded, encoded # Z is returned as well as decoded data poin

def train_autoencoderZ(self, X_train, training_set_size, mini_batch_size
    # Loss function
    loss_function = nn.MSELoss()
    # Optimizer
    optimizer = optim.Adam(self.parameters(), lr = learning_rate)
    # Use Data loader for mini-batches, use training_set_size to select s
    loader = torch.utils.data.DataLoader(X_train[:training_set_size], ba
    for epoch in tqdm(range(n_epochs)):
        for X_batch in loader:
            #print(X_batch)
            # Make forward pass
            decoded, encoded = self.forwardZ(X_batch)
            # Compute loss
            loss = loss_function(decoded, X_batch)

```

```

        # Compute zero-grad
        optimizer.zero_grad()
        # Backprop
        loss.backward()
        # Step
        optimizer.step()
        # Save losses
        self.train_losses.append(loss.item())
    # Save outputs for later
    self.outputs.append((epoch, X_batch, decoded, encoded))

def gmmGenerate(self, components=2):
    """
    This function...
    - gets the decoded and encoded data points from a trained model.
    - trains a GMM with n components (default 2) on the one-dimensional
    - samples generated, encoded data points from the GMM.
    - decodes these new data points via the AE.
    - returns the reconstructed (AE-encoded and -decoded) and the generated
    """
    #Get encoded and reconstructed data points:
    _, _, reconstructed, encoded = self.outputs[-1]
    z_ = encoded[:,0].detach().numpy()

    #Fit GMM to code dist:
    z = z_.reshape(-1, 1)
    gmm = GaussianMixture(n_components = components).fit(z)

    #Sample from GMM:
    z_samples = gmm.sample(n_samples= len(z_))
    z_samples = z_samples[0] #ignore y labels in columns [1]
    z_samples = torch.tensor(z_samples, dtype=torch.float32)

    # Decode generated data:
    for layer in self.decoder:
        z_samples = layer(z_samples)
    generated = z_samples

    return reconstructed, generated

''' MMD Autoencoder training for Exercise 2 '''

def train_mmd_autoencoder(self, X_train, training_set_size, mini_batch_size,
                           mmd_kernel, mmd_bandwidths, lambda_mmd):
    # Loss function
    loss_function = nn.MSELoss()
    # Optimizer
    optimizer = optim.Adam(self.parameters(), lr = learning_rate)
    # Use Data loader for mini-batches, use training_set_size to select subset
    loader = torch.utils.data.DataLoader(X_train[:training_set_size], batch_size=mini_batch_size)
    for epoch in tqdm(range(n_epochs)):
        for X_batch in loader:
            # Make forward pass
            decoded = self.forward(X_batch)
            # Compute reconstruction loss
            recon_loss = loss_function(decoded, X_batch)

```

```

        # Compute MMD loss
        mmd_loss = 0
        for h in mmd_bandwidths:
            mmd_loss += self.compute_mmd(X_batch, h = h, kernel = mn
        # Total loss
        loss = recon_loss + lambda_mmd * mmd_loss
        # Compute zero-grad
        optimizer.zero_grad()
        # Backprop
        loss.backward()
        # Step
        optimizer.step()
        # Save losses
        self.train_losses.append(loss.item())
    # Save outputs for later
    self.outputs.append((epoch, X_batch, decoded))

# Loss function MMD^2
def compute_mmd(self, X_train, h = 1, kernel = "squared_exponential"):
    N = X_train.shape[0]
    # Sample from standard normal
    Z = torch.randn((N, self.bottleneck_size))
    # Get codes from encoder
    codes = X_train
    for layer in self.encoder:
        codes = layer(codes)
    # Calculate distance matrices (vectorized)
    D_XX = self.calculate_distance_matrix(codes, codes)
    D_XZ = self.calculate_distance_matrix(codes, Z)
    D_ZZ = self.calculate_distance_matrix(Z, Z)
    # Calculate kernels
    if kernel == "squared_exponential":
        K_XX = self.squared_exponential(D_XX, h)
        K_XZ = self.squared_exponential(D_XZ, h)
        K_ZZ = self.squared_exponential(D_ZZ, h)
    elif kernel == "inverse_multi_quadratic":
        K_XX = self.inverse_multi_quadratic(D_XX, h)
        K_XZ = self.inverse_multi_quadratic(D_XZ, h)
        K_ZZ = self.inverse_multi_quadratic(D_ZZ, h)
    # Compute MMD^2
    mmd_sq = (torch.sum(K_XX) - torch.trace(K_XX)) / (N * (N - 1)) + \
        (torch.sum(K_ZZ) - torch.trace(K_ZZ)) / (N * (N - 1)) - \
        2 * torch.sum(K_XZ) / (N * N)

    return mmd_sq

def calculate_distance_matrix(self, A, B):
    #  $D_{ij} = ||A_i - B_j||^2 = ||A_i||^2 + ||B_j||^2 - 2 A_i @ B_j.T$ 
    # A_sq in (N_A, 1)
    A_sq = torch.sum(A ** 2, axis = 1, keepdims = True)
    # B_sq in (1, N_B)
    B_sq = torch.sum(B ** 2, axis = 1, keepdims = True).T # .T otherwise
    AB_T = A @ B.T
    D_AB = A_sq + B_sq - 2 * AB_T
    return D_AB

```

```
def squared_exponential(self, D_AB, h):
    return exp(- D_AB / (h ** 2))

def inverse_multi_quadratic(self, D_AB, h):
    # epsilon for numerical stability
    epsilon = 1e-8
    return 1 / sqrt(1 + D_AB / (h ** 2) + epsilon)
```

1. Investigate systematically different hyperparameters:

Starting from `(training_set_size, mini_batch_size, n_epochs, lr, hidden_size, layers) = (1000, 32, 10, 0.001, 4, 2)`, we want to test the influence of the different parameters. We'll compute the Mean Squared Error (MSE) to determine the performance of the different parameter configurations. Since a 6 dimensional phase space is very impractical to explore and perform a grid search for the optimal solution, we'll explore every axis individually and from there converge onto a good solution.

```
In [15]: # 1 Investigate the influence of training set size given fixed other hyperpa
hyperparameters_tss = [(1000, 32, 10, 0.001, 4, 2), # training_set_size, min
                        (10000, 32, 10, 0.001, 4, 2),
                        (50000, 32, 10, 0.001, 4, 2),
                        (100000, 32, 10, 0.001, 4, 2)]

results_tss = []
for params in hyperparameters_tss:
    print(f"Training with training set size: {params[0]}")
    # Create model
    model_tss = Autoencoder(input_size=2,
                            bottleneck_size=1,
                            hidden_size=params[4],
                            layers=params[5])

    # Train model
    model_tss.train_autoencoder(X_train,
                                training_set_size=params[0],
                                mini_batch_size=params[1],
                                n_epochs=params[2],
                                learning_rate=params[3])

    # Save results
    results_tss.append((params, model_tss.train_losses, model_tss.outputs[-1]))
```

Training with training set size: 1000

100%|██████████| 10/10 [00:00<00:00, 49.60it/s]

Training with training set size: 10000

100%|██████████| 10/10 [00:01<00:00, 5.20it/s]

Training with training set size: 50000

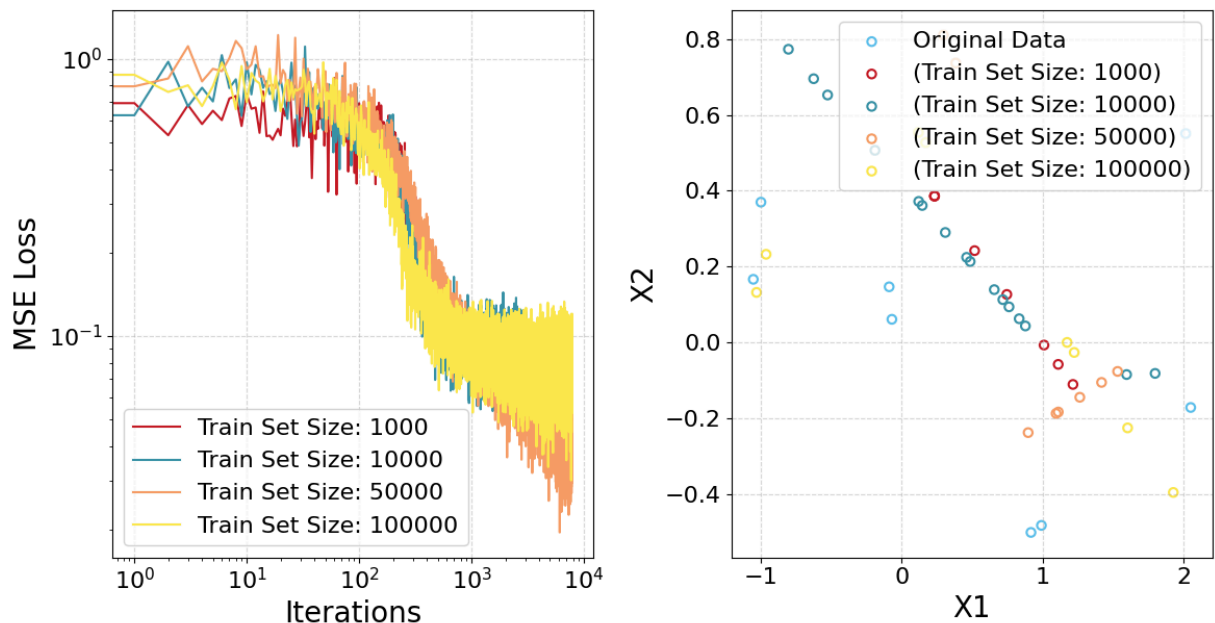
100%|██████████| 10/10 [00:04<00:00, 2.05it/s]

Training with training set size: 100000

100%|██████████| 10/10 [00:04<00:00, 2.08it/s]


```
In [16]: # Plot MSE loss for different training set sizes
title = "Autoencoder Training: Influence of Training Set Size"
plot1_1(model_tss, results_tss, title, label="Train Set Size", label_index=
```

Autoencoder Training: Influence of Training Set Size



Simply increasing the test size does obviously no good if the underlying network architecture is poor.

```
In [17]: # 1 Investigate the influence of number of epochs given fixed other hyperpar
hyperparameters_n_epochs = [(2000, 32, 10, 0.001, 4, 2), # training_set_size
                             (2000, 32, 100, 0.001, 4, 2),
                             (2000, 32, 1000, 0.001, 4, 2),
                             (2000, 32, 5000, 0.001, 4, 2)]

results_n_epochs = []
for params in hyperparameters_n_epochs:
    print(f"Training with training set size: {params[0]}, epochs: {params[2]}")
    # Create model
    model_n_epochs = Autoencoder(input_size=2,
                                 bottleneck_size=1,
                                 hidden_size=params[4],
                                 layers=params[5])

    # Train model
    model_n_epochs.train_autoencoder(X_train,
                                     training_set_size=params[0],
                                     mini_batch_size=params[1],
                                     n_epochs=params[2],
                                     learning_rate=params[3])

    # Save results
    results_n_epochs.append((params, model_n_epochs.train_losses, model_n_epochs.test_losses))
```

Training with training set size: 2000, epochs: 10

100%|██████████| 10/10 [00:00<00:00, 25.45it/s]

Training with training set size: 2000, epochs: 100

100%|██████████| 100/100 [00:03<00:00, 25.35it/s]

Training with training set size: 2000, epochs: 1000

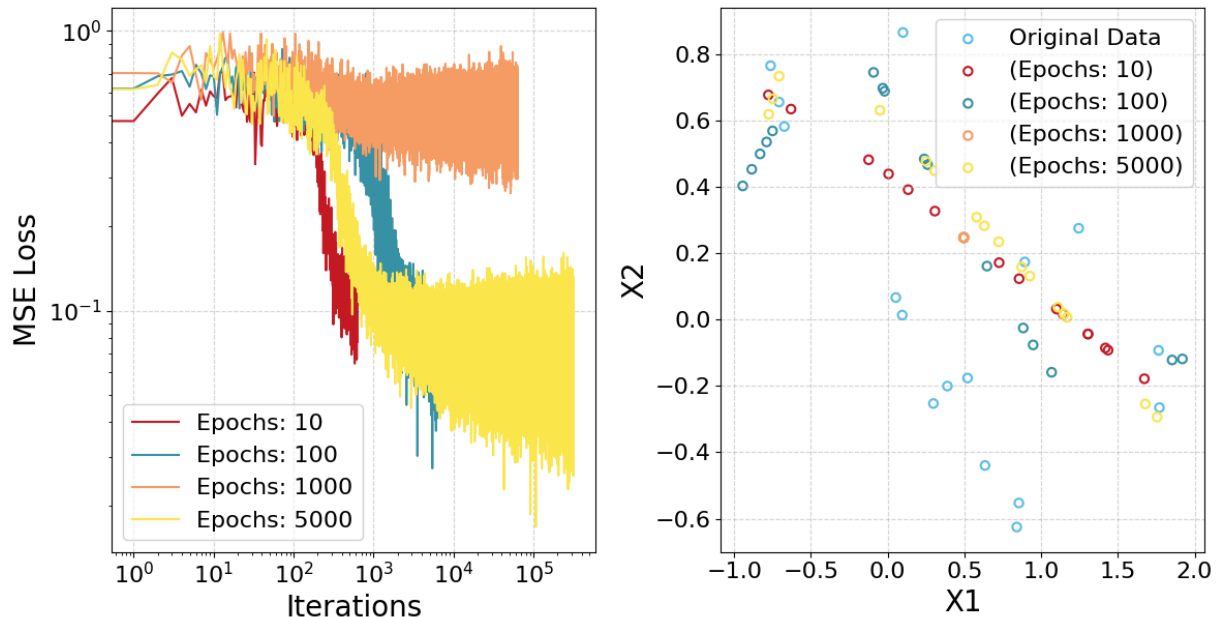
100%|██████████| 1000/1000 [00:39<00:00, 25.44it/s]

Training with training set size: 2000, epochs: 5000

100%|██████████| 5000/5000 [03:14<00:00, 25.65it/s]

```
In [18]: # Plot MSE loss for different number of epochs
title = "Autoencoder Training: Influence of Number of Epochs"
plot1_1(model_n_epochs, results_n_epochs, title, label="Epochs", label_index
```

Autoencoder Training: Influence of Number of Epochs



Increasing the training iterations by brute force provides mixed and inconsistent results. In the left diagram, one can see the MSE loss shows a reduction between 100 and 1000 training iterations and the reconstructed data appears to be in the same order of magnitude as the data, in contrast to the sample where we simply increased the number of training points. Even the best results show a very linear trend and fail to capture the profile of the two moon samples. In some cases, the models trained for 1000 or 5000 epochs approximate the curved behavior with extra diagonal lines. However, our hypothesis is that this will not improve much until we expand the size of our network, as it currently only has 2 layers of dimension 4. Let's first test whether this can be improved by increasing the number of samples the model trains simultaneously from `mini_batch_size = 32 -> 256`.

```
In [19]: # 1 Investigate the influence of mini-batch-size given fixed other hyperpara
hyperparameters_mbs = [(2000, 32, 1000, 0.001, 4, 2), # training_set_size, n
                        (2000, 64, 1000, 0.001, 4, 2),
                        (2000, 128, 1000, 0.001, 4, 2),
                        (2000, 256, 1000, 0.001, 4, 2)]

results_mbs = []
```

```

for params in hyperparameters_mbs:
    print(f"Training with training set size: {params[0]}, n_epochs: {params[1]}")
    # Create model
    model_mbs = Autoencoder(input_size=2,
                            bottleneck_size=1,
                            hidden_size=params[4],
                            layers=params[5])

    # Train model
    model_mbs.train_autoencoder(X_train,
                                training_set_size=params[0],
                                mini_batch_size=params[1],
                                n_epochs=params[2],
                                learning_rate=params[3])

    # Save result
    results_mbs.append((params, model_mbs.train_losses, model_mbs.outputs[-1]))

```

Training with training set size: 2000, n_epochs: 1000, mini batch size: 32

100%|██████████| 1000/1000 [00:38<00:00, 25.77it/s]

Training with training set size: 2000, n_epochs: 1000, mini batch size: 64

100%|██████████| 1000/1000 [00:21<00:00, 46.70it/s]

Training with training set size: 2000, n_epochs: 1000, mini batch size: 128

100%|██████████| 1000/1000 [00:12<00:00, 79.91it/s]

Training with training set size: 2000, n_epochs: 1000, mini batch size: 256

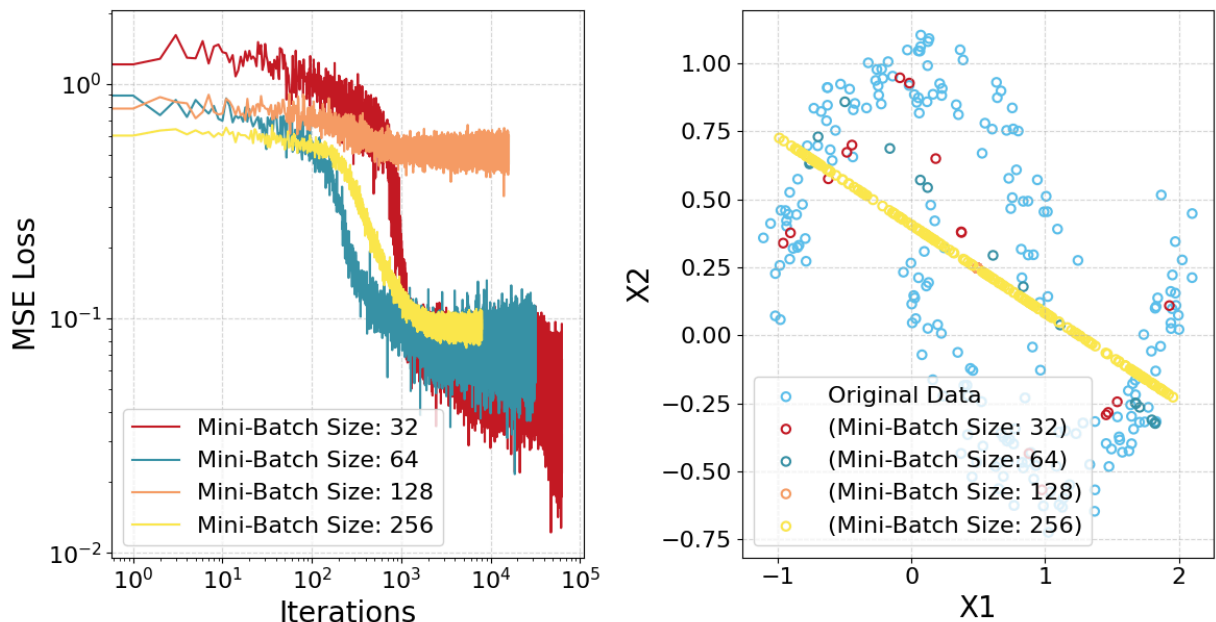
100%|██████████| 1000/1000 [00:10<00:00, 97.43it/s]

```

In [20]: # Plot MSE loss for different mini-batch-sizes
title = "Autoencoder Training: Influence of Mini-Batch Size"
plot1_1(model_mbs, results_mbs, title, label="Mini-Batch Size", label_index=

```

Autoencoder Training: Influence of Mini-Batch Size



As expected, increasing the test-points in every training iteration by using larger mini-batches provides inconsistent or at best marginally better results. Next we will test different network sizes. Since the data is relatively simple, increasing the number of

layers beyond a certain extent will likely not provide significant results. Instead, it could cause issues due to vanishing gradients

```
In [21]: # 1 Investigate the influence of network-sizes given fixed other hyperparameters
hyperparameters_lyrs = [(2000, 128, 2000, 0.001, 4, 2), # training_set_size,
                        (2000, 128, 2000, 0.001, 4, 3),
                        (2000, 128, 2000, 0.001, 4, 4),
                        (2000, 128, 2000, 0.001, 4, 8)]

results_lyrs = []
for params in hyperparameters_lyrs:
    print(f"Training with training set size: {params[0]}, n_epochs: {params[2]}")
    # Create model
    model_lyrs = Autoencoder(input_size=2,
                             bottleneck_size=1,
                             hidden_size=params[4],
                             layers=params[5])

    # Train model
    model_lyrs.train_autoencoder(X_train,
                                  training_set_size=params[0],
                                  mini_batch_size=params[1],
                                  n_epochs=params[2],
                                  learning_rate=params[3])

    # Save result
    results_lyrs.append((params, model_lyrs.train_losses, model_lyrs.outputs
```

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, layers: 2

100%|██████████| 2000/2000 [00:24<00:00, 81.28it/s]

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, layers: 3

100%|██████████| 2000/2000 [00:29<00:00, 67.55it/s]

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, layers: 4

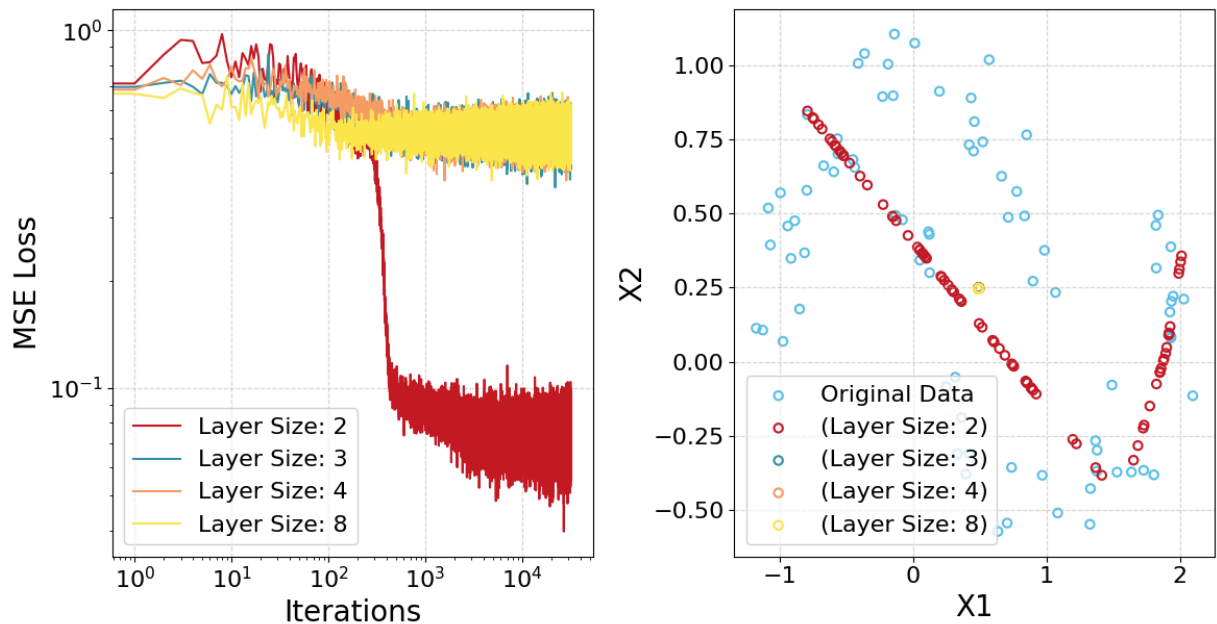
100%|██████████| 2000/2000 [00:34<00:00, 58.37it/s]

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, layers: 8

100%|██████████| 2000/2000 [00:53<00:00, 37.34it/s]

```
In [22]: # Plot MSE loss for different layer-sizes
title = "Autoencoder Training: Influence of Layer Size"
plot1_1(model_lyrs, results_lyrs, title, label="Layer Size", label_index= 5
```

Autoencoder Training: Influence of Layer Size



Indeed, increasing the number of layers doesn't correlate with better performance.

However, wider layers are more robust against vanishing layers and learn more features such as the curves of the two moons, the gap between them, etc. Likely this is the defining parameter between a model which kind of understands what it should be doing and one which does what it needs to do.

```
In [23]: # 1 Investigate the influence of network-sizes given fixed other hyperparameters
hyperparameters_hsize = [(2000, 128, 2000, 0.001, 4, 4), # training_set_size
                          (2000, 128, 2000, 0.001, 8, 4),
                          (2000, 128, 2000, 0.001, 16, 4),
                          (2000, 128, 2000, 0.001, 32, 4)]

results_hsize = []
for params in hyperparameters_hsize:
    print(f"Training with training set size: {params[0]}, n_epochs: {params[2]}")
    # Create model
    model_hsize = Autoencoder(input_size=2,
                              bottleneck_size=1,
                              hidden_size=params[4],
                              layers=params[5])

    # Train model
    model_hsize.train_autoencoder(X_train,
                                  training_set_size=params[0],
                                  mini_batch_size=params[1],
                                  n_epochs=params[2],
                                  learning_rate=params[3])

    # Save result
    results_hsize.append((params, model_hsize.train_losses, model_hsize.outp
```

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, hidden size: 4, layers: 4

100%|██████████| 2000/2000 [00:34<00:00, 57.98it/s]

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, hidden size: 8, layers: 4

100%|██████████| 2000/2000 [00:34<00:00, 57.16it/s]

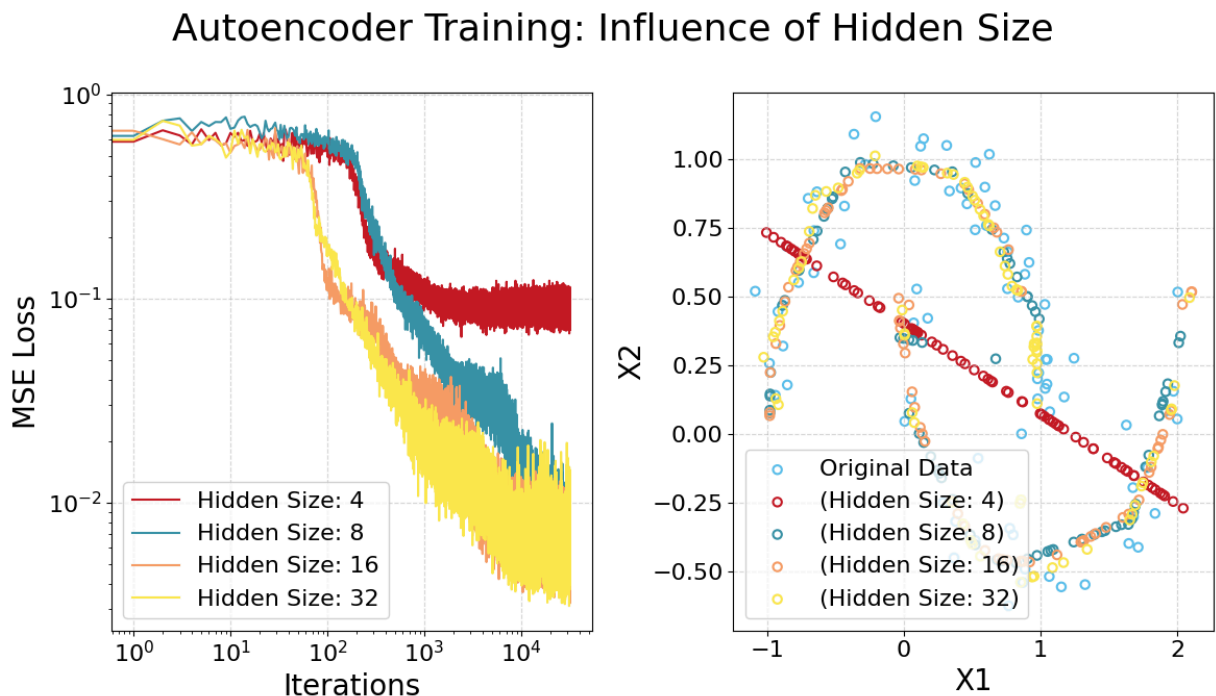
Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, hidden size: 16, layers: 4

100%|██████████| 2000/2000 [00:35<00:00, 56.09it/s]

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, hidden size: 32, layers: 4

100%|██████████| 2000/2000 [00:40<00:00, 48.81it/s]

```
In [24]: # Plot MSE loss for different hidden-sizes
title = "Autoencoder Training: Influence of Hidden Size"
plot1_1(model_hsize, results_hsize, title, label="Hidden Size", label_index=
```



Indeed! Upon visual inspection, the models are learning the nuances of the two moons data, in particularl adapting to the gap between the two moons (hidden size: 32) and following the curves more accurately (hidden size: 8+). Now, we would like to refine the rest of the hyperparameters. For example, the Loss diagram shows that there is still more room to let the model converge, so we should increase the number of iterations it trains for.

```
In [25]: # 1 Investigate the influence of network-sizes given fixed other hyperparamete
hyperparameters_refine_n_epochs = [(2000, 128, 2000, 0.001, 16, 4), # traini
                                   (2000, 128, 4000, 0.001, 16, 4),
                                   (2000, 128, 8000, 0.001, 16, 4),
                                   (2000, 128, 10000, 0.001, 16, 4)]

results_refine_n_epochs = []
for params in hyperparameters_refine_n_epochs:
    print(f"Training with training set size: {params[0]}, n_epochs: {params[1]}")
    # Create model
    model_refine_n_epochs = Autoencoder(input_size=2,
```

```

bottleneck_size=1,
hidden_size=params[4],
layers=params[5])

# Train model
model_refine_n_epochs.train_autoencoder(X_train,
                                         training_set_size=params[0],
                                         mini_batch_size=params[1],
                                         n_epochs=params[2],
                                         learning_rate=params[3])

# Save result
results_refine_n_epochs.append((params, model_refine_n_epochs.train_loss

```

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, hidden size: 16, layers: 4

100%|██████████| 2000/2000 [00:36<00:00, 55.09it/s]

Training with training set size: 2000, n_epochs: 4000, mini batch size: 128, hidden size: 16, layers: 4

100%|██████████| 4000/4000 [01:11<00:00, 55.75it/s]

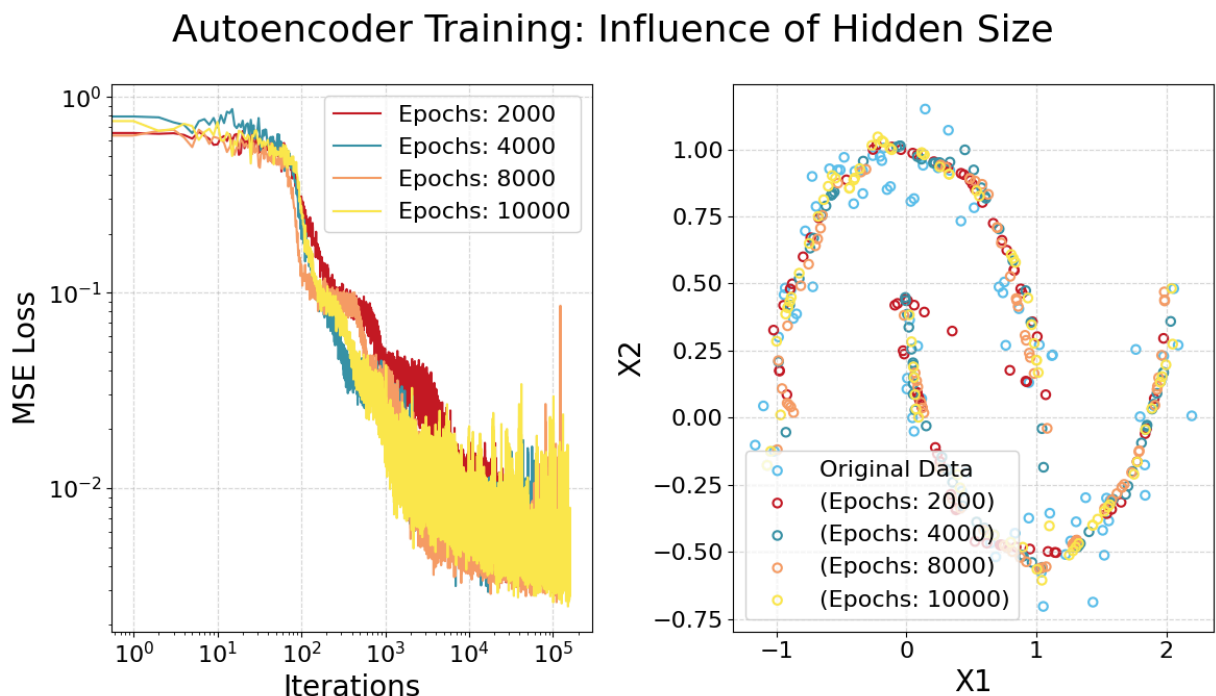
Training with training set size: 2000, n_epochs: 8000, mini batch size: 128, hidden size: 16, layers: 4

100%|██████████| 8000/8000 [02:25<00:00, 55.13it/s]

Training with training set size: 2000, n_epochs: 10000, mini batch size: 128, hidden size: 16, layers: 4

100%|██████████| 10000/10000 [03:00<00:00, 55.53it/s]

In [26]: `# Plot MSE loss for different hidden-sizes`
`title = "Autoencoder Training: Influence of Hidden Size"`
`plot1_1(model_refine_n_epochs, results_refine_n_epochs, title, label="Epochs"`



There's no noticeable improvement beyond 2000-4000 training epochs. Next we'll try softer and more aggressive learning rates.


```
In [27]: # 1 Investigate the influence of learning rates given fixed other hyperparameters
hyperparameters_refine_lr = [(2000, 128, 4000, 0.1, 16, 4), # training_set_size, mini_batch_size, n_epochs, learning_rate, hidden_size, layers]
                        (2000, 128, 4000, 0.01, 16, 4),
                        (2000, 128, 4000, 0.001, 16, 4),
                        (2000, 128, 4000, 0.0001, 16, 4)]

results_refine_lr = []
for params in hyperparameters_refine_lr:
    print(f"Training with training set size: {params[0]}, n_epochs: {params[2]}, mini batch size: {params[1]}, hidden size: {params[4]}, layers: {params[5]}")
    # Create model
    model_refine_lr = Autoencoder(input_size=2,
                                  bottleneck_size=1,
                                  hidden_size=params[4],
                                  layers=params[5])

    # Train model
    model_refine_lr.train_autoencoder(X_train,
                                       training_set_size=params[0],
                                       mini_batch_size=params[1],
                                       n_epochs=params[2],
                                       learning_rate=params[3])

    # Save result
    results_refine_lr.append((params, model_refine_lr.train_losses, model_refine_lr.train_losses))
```

Training with training set size: 2000, n_epochs: 4000, mini batch size: 128, hidden size: 16, layers: 4

100%|██████████| 4000/4000 [01:11<00:00, 56.11it/s]

Training with training set size: 2000, n_epochs: 4000, mini batch size: 128, hidden size: 16, layers: 4

100%|██████████| 4000/4000 [01:11<00:00, 56.07it/s]

Training with training set size: 2000, n_epochs: 4000, mini batch size: 128, hidden size: 16, layers: 4

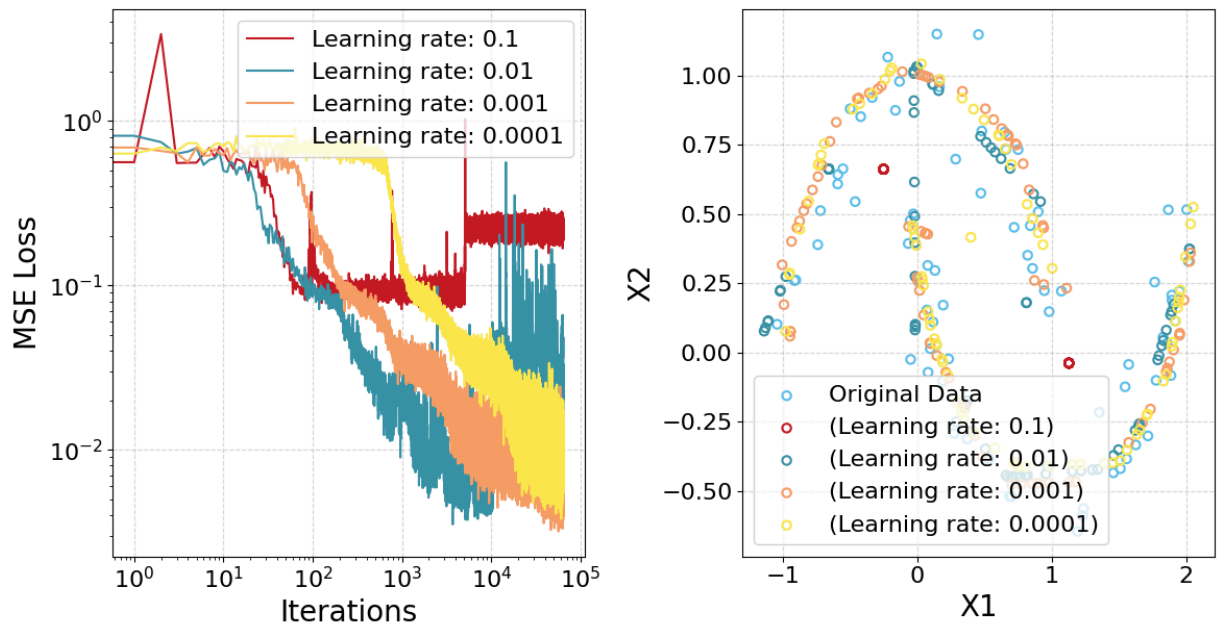
100%|██████████| 4000/4000 [01:11<00:00, 56.22it/s]

Training with training set size: 2000, n_epochs: 4000, mini batch size: 128, hidden size: 16, layers: 4

100%|██████████| 4000/4000 [01:10<00:00, 56.40it/s]

```
In [28]: # Plot MSE loss for different learning rates
title = "Autoencoder Training: Influence of Learning Rate"
plot1_1(model_refine_lr, results_refine_lr, title, label="Learning rate", label_pos="bottom")
```


Autoencoder Training: Influence of Learning Rate



An aggressive learning rate of 0.1 seems to break down at some point, while smaller learning rates do comparatively well and similar to each other, except that a learning rate of 0.0001 simply takes longer to get there compared to 0.01 and 0.001. One could implement a learning schedule, but I think for our purposes with simple data this would be overkill. Instead, we'll stick to a learning rate between 0.01 - 0.001. Now let's try to economize and cut corners where we may have initially overcompensated. For example, we chose 4 hidden layers because it worked best when there were 4 neurons each. Then we settled on 16 neurons each without adjusting the number of layers.

```
In [29]: # 1 Investigate the influence of learning rates given fixed other hyperparameters
hyperparameters_refine_hs_lyrs = [(2000, 128, 2000, 0.005, 16, 4),
                                   (2000, 128, 2000, 0.005, 12, 4),
                                   (2000, 128, 2000, 0.005, 16, 3), # training
                                   (2000, 128, 2000, 0.005, 12, 3),
                                   (2000, 128, 2000, 0.005, 16, 2),
                                   (2000, 128, 2000, 0.005, 12, 2),
                                   #(2000, 128, 2000, 0.005, 8, 2),
                                   #(2000, 128, 2000, 0.005, 6, 2)
                                   ]

results_refine_hs_lyrs = []
for params in hyperparameters_refine_hs_lyrs:
    print(f"Training with training set size: {params[0]}, n_epochs: {params[1]}")
    # Create model
    model_refine_hs_lyrs = Autoencoder(input_size=2,
                                       bottleneck_size=1,
                                       hidden_size=params[4],
                                       layers=params[5])

    # Train model
    model_refine_hs_lyrs.train_autoencoder(X_train,
                                           training_set_size=params[0],
```

```

        mini_batch_size=params[1],
        n_epochs=params[2],
        learning_rate=params[3])

# Save result
results_refine_hs_lyrs.append((params, model_refine_hs_lyrs.train_losses

```

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, hidden size: 16, layers: 4

100%|██████████| 2000/2000 [00:34<00:00, 57.41it/s]

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, hidden size: 12, layers: 4

100%|██████████| 2000/2000 [00:34<00:00, 57.93it/s]

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, hidden size: 16, layers: 3

100%|██████████| 2000/2000 [00:29<00:00, 67.30it/s]

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, hidden size: 12, layers: 3

100%|██████████| 2000/2000 [00:29<00:00, 67.80it/s]

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, hidden size: 16, layers: 2

100%|██████████| 2000/2000 [00:24<00:00, 81.57it/s]

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128, hidden size: 12, layers: 2

100%|██████████| 2000/2000 [00:24<00:00, 82.20it/s]

```

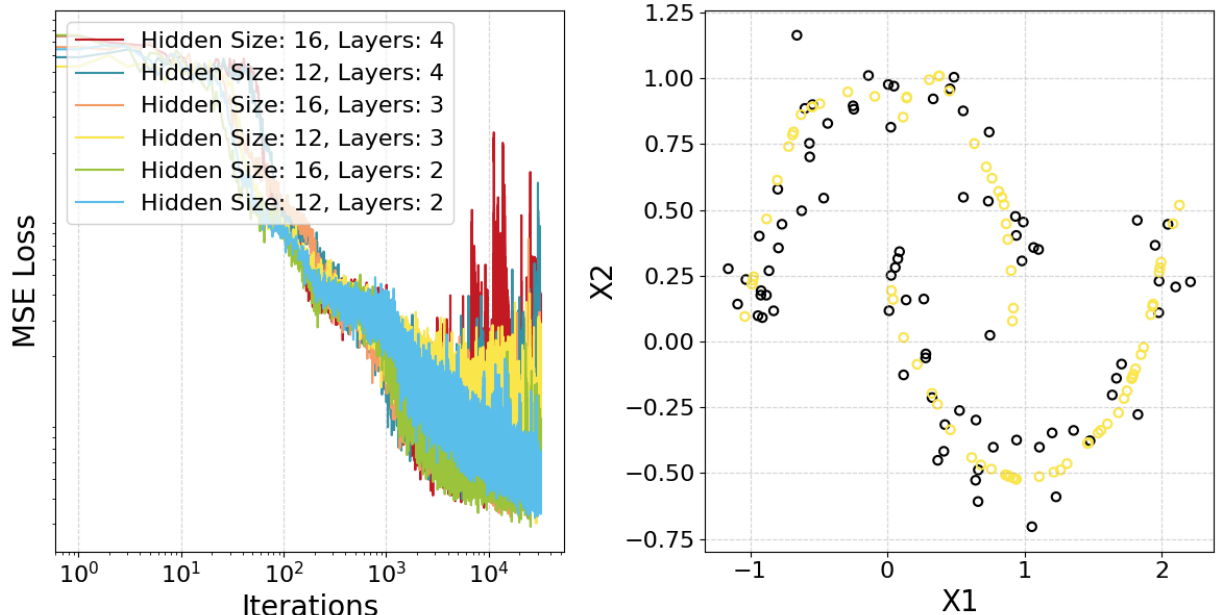
In [30]: # Plot MSE loss for different learning rates
fig, axes = plt.subplots(1, 2)
fig.suptitle("Autoencoder Training: Influence of Layers and hidden size", fo
# Plot training losses
for res in results_refine_hs_lyrs:
    params, train_losses, _ = res
    axes[0].plot(train_losses, label=f"Hidden Size: {params[4]}, Layers: {pa
axes[0].set_xlabel("Iterations")
axes[0].set_ylabel("MSE Loss")
axes[0].loglog('log')
axes[0].legend()

# Plot final reconstruction
# Get final output
final_epoch, X_batch, decoded = results_refine_hs_lyrs[-1][2]
axes[1].scatter(X_batch[:,0].detach().numpy(), X_batch[:,1].detach().numpy()
                label="Original Data",
                color = "black")
for i in range(len(results_refine_hs_lyrs)):
    if i in [1, 2, 0, 4, 5]:
        continue
    final_epoch, X_batch, decoded = results_refine_hs_lyrs[i][2]
    axes[1].scatter(decoded[:,0].detach().numpy(), decoded[:,1].detach().num
                    label=f"(Hidden size: {results_refine_hs_lyrs[i][0][4]},
                    alpha=1,
                    color = colors[i])
axes[1].set_xlabel("X1")
axes[1].set_ylabel("X2")
#axes[1].legend()

```

```
plt.tight_layout()
plt.show()
```

Autoencoder Training: Influence of Layers and hidden size



While reducing the number of neurons and layers still yields recognizable results, the lines get smoother and smoother for a wider and slightly deeper architecture. We find the best combination of depth and width to be 3 layers of 16 neurons each.

1.2 Reconstruction with the same or different data sets

To evaluate the optimized parameters (`training_set_size`, `mini_batch_size`, `n_epochs`, `lr`, `hidden_size`, `layers`) = (2000, 128, 2000, 0.005, 16, 3), we repeat training with the same or different datasets and evaluate the mean squared error and its variance. We start by training the same model again with an identical data set. This is not expected to make a big difference in performance, since no 'additional' information about the distribution is provided.

```
In [7]: params = (2000, 128, 2000, 0.005, 16, 3) # training_set_size, mini_batch_size, n_epochs, lr, hidden_size, layers

#Make a list of Data sets:
#Train the model with the same data set again:
sameSets = [(X_train, "X_train"), (X_train, "X_train"), (X_train, "X_train")]

results_sameSets = []
print(f"Training with training set size: {params[0]}, n_epochs: {params[2]},")
for TS in sameSets:
    print(f"Training set: {TS[1]}")
    # Create model
    model_sameSets = Autoencoder(input_size=2,
                                bottleneck_size=1,
```

```

        hidden_size=params[4],
        layers=params[5])

# Train model
model_sameSets.train_autoencoder(TS[0],
                                training_set_size=params[0],
                                mini_batch_size=params[1],
                                n_epochs=params[2],
                                learning_rate=params[3])

# Save result
results_sameSets.append((TS[1], params, model_sameSets.train_losses, mod

```

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128,
hidden size: 16, layers: 3
Training set: X_train

100%|██████████| 2000/2000 [00:29<00:00, 66.73it/s]

Training set: X_train

100%|██████████| 2000/2000 [00:30<00:00, 66.29it/s]

Training set: X_train

100%|██████████| 2000/2000 [00:30<00:00, 66.35it/s]

```

In [8]: title= "Model training with identical datasets"
        plot1_2(model_sameSets, results_sameSets, title)

```

Set: Mean, Std

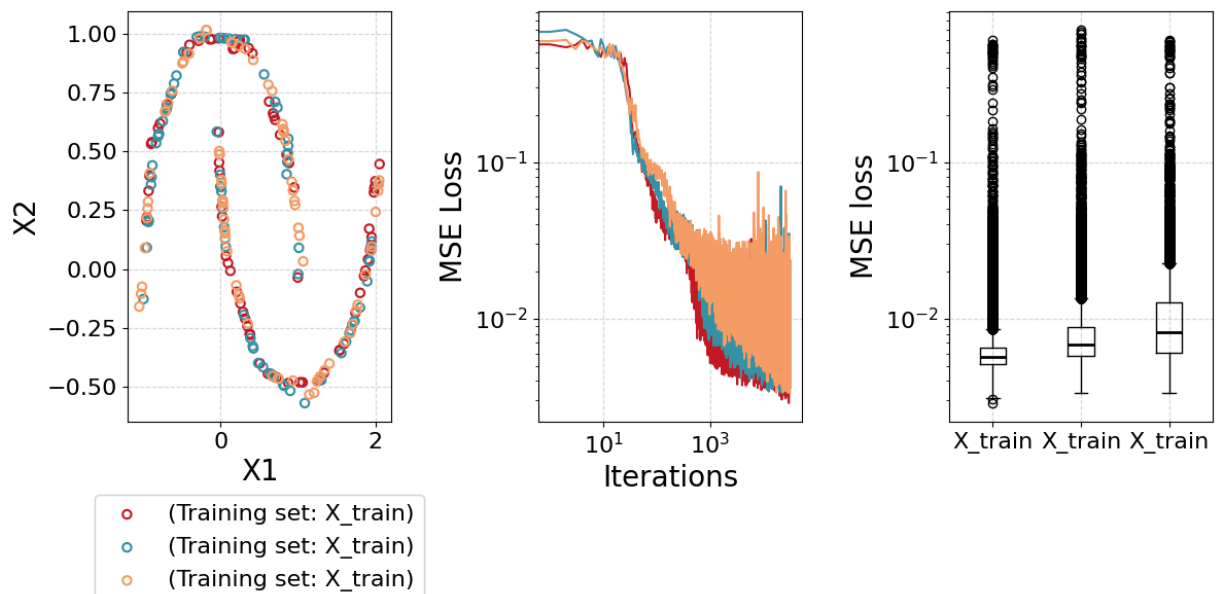
X_train: 0.0071 0.0161

X_train: 0.0088 0.0164

X_train: 0.0110 0.0169

Average MSE over all data sets: 0.0090 with a standard deviation of 0.0016

Model training with identical datasets



Training with the same data set does not make a big difference in reconstruction quality of the models, as expected.

The reconstructed data points look very similar and the MSE losses behave similarly. Overall, a mean MSE loss of ~0.009 is reached with a standard deviation of 0.001. Yet, outliers are possible when retraining the model.

```

In [9]: #Make a list of Data sets:
def makeTS(n_samples=125000, noise=0.1, random_state=42):
    X, y = make_moons(n_samples= n_samples, noise= noise, random_state= random_state)
    X = torch.tensor(X, dtype=torch.float32)
    X_test, X_train = train_test_split(X, test_size=0.2, random_state=42)
    return X_train

n_sets = 3
diffSets = [(X_train, "X_train")]
for i in range(1,n_sets):
    X = makeTS()
    diffSets.append((X, f"X_{i}"))
#diffSets.append((X_train, "X_train"))

# Train the model with different data sets:
results_diffSets = []
print(f"Training with training set size: {params[0]}, n_epochs: {params[2]},")
for TS in diffSets:
    print(f"Training set: {TS[1]}")
    # Create model
    model_diffSets = Autoencoder(input_size=2,
                                bottleneck_size=1,
                                hidden_size=params[4],
                                layers=params[5])

    # Train model
    model_diffSets.train_autoencoder(TS[0], !!! the same model is trained again
                                    training_set_size=params[0],
                                    mini_batch_size=params[1],
                                    n_epochs=params[2],
                                    learning_rate=params[3])

    # Save result
    results_diffSets.append((TS[1], params, model_diffSets.train_losses, model_diffSets.train_losses))

```

Training with training set size: 2000, n_epochs: 2000, mini batch size: 128,
hidden size: 16, layers: 3

Training set: X_train

100%|██████████| 2000/2000 [00:30<00:00, 66.37it/s]

Training set: X_1

100%|██████████| 2000/2000 [00:30<00:00, 66.23it/s]

Training set: X_2

100%|██████████| 2000/2000 [00:30<00:00, 66.15it/s]

```

In [11]: title= "Model training with different datasets"
plot1_2(model_diffSets, results_diffSets, title)

```

Set: Mean, Std

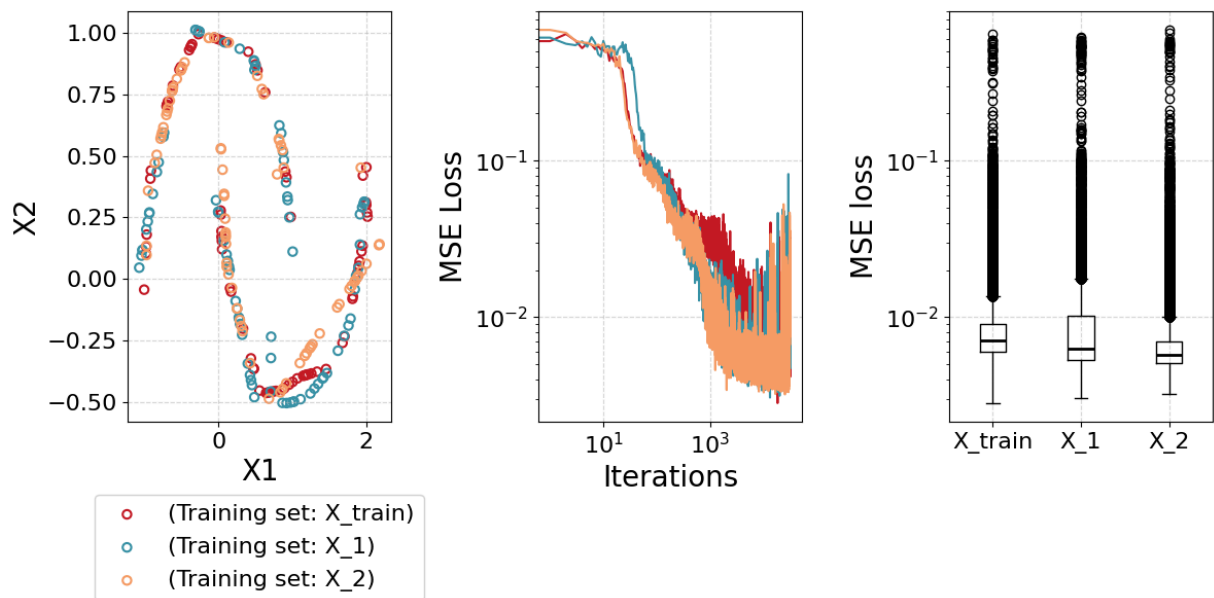
X_train: 0.0098 0.0163

X_1: 0.0096 0.0194

X_2: 0.0080 0.0159

Average MSE over all data sets: 0.0092 with a standard deviation of 0.0008

Model training with different datasets



Training the model with different data sets results in visibly different reconstructed distributions.

However, the MSE losses have similar values and standard deviations (~ 0.009 and ~ 0.001) compared to re-training with identical data sets. The overall shape of two moons is still clearly visible.

1.3 Visualising the code distribution and generating samples

We start by training a model on the original training data with the defined hyperparameters (`params`). We then use train a GaussianMixture Model with 2 components. (More components lead to overfitting of the 1D-data set.) Using this GMM, new samples are generated and decoded.

See Figure below.

```
In [51]: #Train model with ideal parameters (params) and X_train:
# Create model
model_Z = Autoencoder(input_size=2, bottleneck_size=1, hidden_size=params[4])
# Train model
model_Z.train_autoencoderZ(X_train, training_set_size=params[0], mini_batch_size=100,
_, _, decoded, encoded = model_Z.outputs[-1]
z = encoded[:,0].detach().numpy()
x,y = decoded[:,0].detach().numpy(), decoded[:,1].detach().numpy()
```

100% |██████████| 2000/2000 [00:32<00:00, 62.47it/s]

```
In [52]: #Fit GMM:
z_ = z.reshape(-1, 1)
gmm = GaussianMixture(n_components = components).fit(z_)
```

```

#Get parameters
mean = gmm.means_
covs = gmm.covariances_
weights = gmm.weights_

#Generate samples:
x_hat, x_gen = model_Z.gmmGenerate(2) #see class implementation

```

In [53]: *#Visualize Code distribution:*

```

#Calculate Histogram parameters:
bin_counts, bin_edges = np.histogram(z)
bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2
bin_widths = np.diff(bin_edges)
bin_density = bin_counts/(np.sum(bin_counts)*bin_widths) #  $P(\text{bin}) = N1 / (H \times W)$ 

fig, axes = plt.subplots(1, 2, figsize=(20,10))
fig.suptitle("Generating samples from the encoded data points", fontsize=26)

#Plot Density Histogram with GMM:
a = 0
axes[a].bar(bin_centers, bin_density, width=bin_widths, align='center', color=colors[a])
axes[a].set_title("Code distribution - Histogram & GMM", fontsize=16)
z_axis = np.arange((np.min(z)*1.5), (np.max(z)*1.5), 0.1)
y_all = np.zeros((len(z), len(z_axis)))
for n in range(components):
    y_axis = norm.pdf(z_axis, float(mean[n][0]), np.sqrt(float(covs[n][0][0])))
    axes[a].plot(z_axis, y_axis, lw=3, ls='dashed', label=f"Gaussian #{n+1}")
    y_all += y_axis
axes[a].plot(z_axis, y_all[0], lw=3, c='C2', label=f"GMM ({n} components)")
axes[a].set_xlabel(f"Z", fontsize=20)
axes[a].set_ylabel(f"Density", fontsize=20)
axes[a].legend(loc='upper center', bbox_to_anchor=(0.5, -0.15), ncols=3)

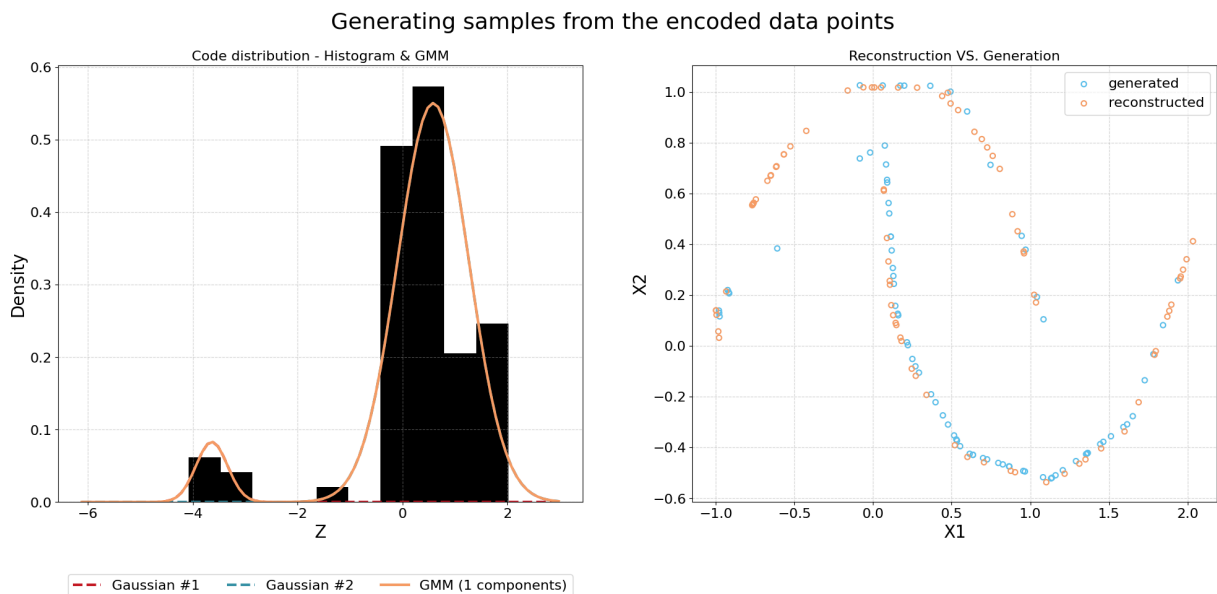
...

# Plot final reconstruction
axes[0].scatter(x,y, color = colors[0])
axes[0].set_xlabel("X1")
axes[0].set_ylabel("X2")
#axes[0].legend(loc='upper center', bbox_to_anchor=(0.5, -0.15))
axes[0].set_title("Reconstructed data", fontsize=16)
...

# Plot final reconstruction
b = 1
axes[b].scatter(x_gen[:,0].detach().numpy(), x_gen[:,1].detach().numpy(), label=f"generated", color = colors[1])
#axes[b].scatter(x_hat[:,0].detach().numpy(), x_hat[:,1].detach().numpy(), label=f"reconstructed", color = colors[2])
axes[b].scatter(x, y, label=f"reconstructed", color = colors[2])
axes[b].set_xlabel("X1")
axes[b].set_ylabel("X2")
axes[b].legend()
axes[b].set_title("Reconstruction VS. Generation", fontsize=16)

plt.tight_layout()
plt.show()

```



Notably, the largest bins in the code distribution are usually between the z-coordinates roughly relate to the maximum number of data point in the true distribution.

The quality of the generated data is visibly worse than the reconstructed data with more outliers and point connecting the two moons horizontally, in some cases.

Still, it also heavily depends on how accurate the reconstructed data points fit the true distribution.

1.4 Testing AE with larger noise

We use the original autoencoder set-up (`params`) with the original training set `X_train` with noise level 0.1. Then, we evaluate the AE performance with a noisy test set (noise level 0.2).

The reconstruction quality at a higher noise level is visibly worse. This makes sense, as the model is not trained for data sets with a larger variance and will hence produce inaccurate results.

```
In [77]: #Train model with ideal parameters (params) and X_train (noise 0.1):
# Create model
model_tryNoise = Autoencoder(input_size=2, bottleneck_size=1, hidden_size=pa
# Train model
model_tryNoise.train_autoencoder(X_train,training_set_size=params[0], mini_t
_, _, decoded = model_tryNoise.outputs[-1]
#x,y = decoded[:,0].detach().numpy(), decoded[:,1].detach().numpy()
```

100% |██████████| 2000/2000 [00:32<00:00, 62.37it/s]

```
In [82]: X_Noisy = makeTS(noise=0.2)
X_Noisy = X_Noisy.detach().clone()
```



```

Noise = model_tryNoise.forward(X_Noisy)
NoNoise = model_tryNoise.forward(X_train)

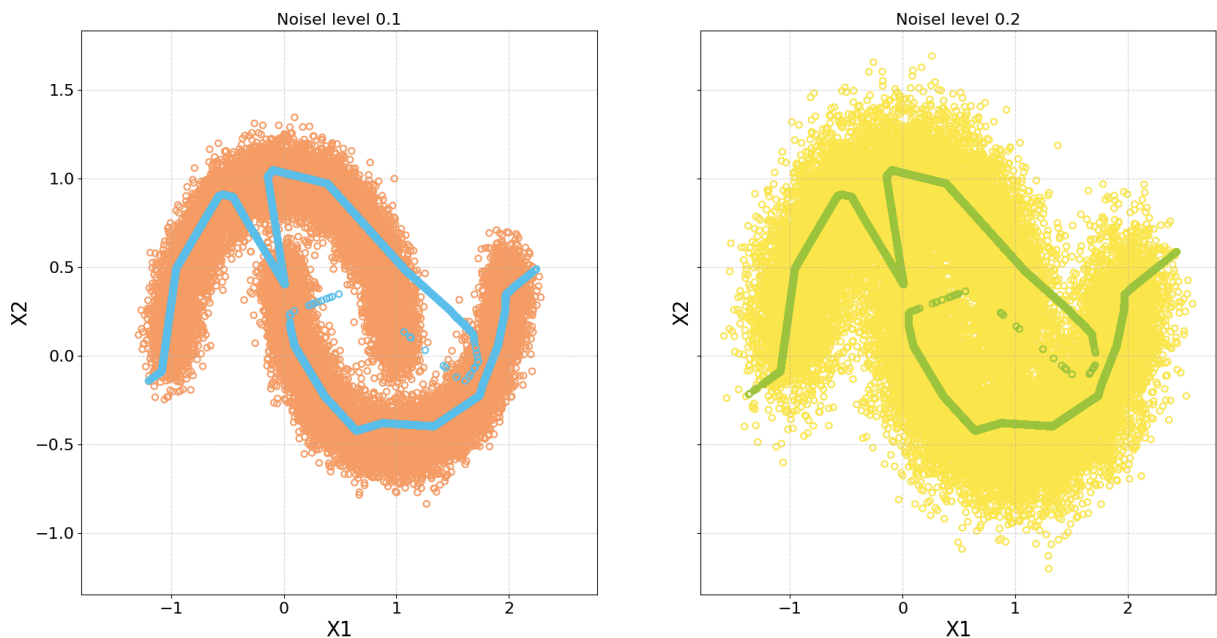
# Plot:
fig, axes = plt.subplots(1, 2, figsize= (20,10), sharex= True, sharey= True)
fig.suptitle("Original Model tested at two noise levels", fontsize=26)

axes[0].scatter(X_train[:,0].detach().numpy(), X_train[:,1].detach().numpy())
axes[0].scatter(NoNoise[:,0].detach().numpy(), NoNoise[:,1].detach().numpy())
axes[0].set_xlabel("X1")
axes[0].set_ylabel("X2")
axes[0].set_title("Noise level 0.1", fontsize= 16)
axes[1].scatter(X_Noisy[:,0].detach().numpy(), X_Noisy[:,1].detach().numpy())
axes[1].scatter(Noise[:,0].detach().numpy(), Noise[:,1].detach().numpy(), label=1)
axes[1].set_xlabel("X1")
axes[1].set_ylabel("X2")
axes[1].set_title("Noise level 0.2", fontsize= 16)
#plt.title(, fontsize= 16)

#plt.tight_layout()
plt.show()

```

Original Model tested at two noise levels



1.5 Training AE with larger noise

```

In [71]: #Train model with ideal parameters (params) and X_train (noise 0.1):
# Create model
model_Noisy = Autoencoder(input_size=2, bottleneck_size=1, hidden_size=params[0])
# Train model
model_Noisy.train_autoencoder(X_Noisy, training_set_size=params[0], mini_batch_size=100, NoiseT = model_Noisy.outputs[-1])
#x,y = decoded[:,0].detach().numpy(), decoded[:,1].detach().numpy()

```

100% | 2000/2000 [00:31<00:00, 64.15it/s]

```

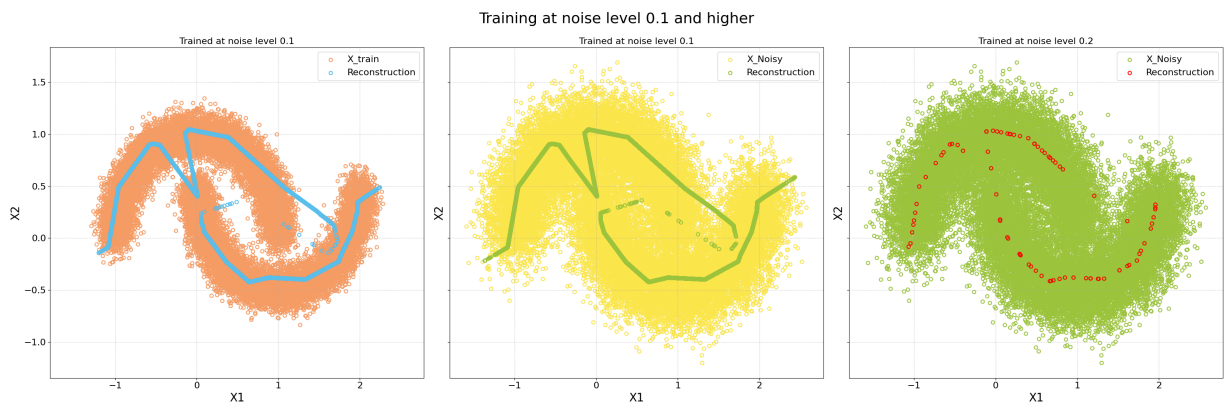
In [90]: # Plot:
NoiseT = decoded
fig, axes = plt.subplots(1, 3, figsize= (30,10), sharex= True, sharey= True)
fig.suptitle("Training at noise level 0.1 and higher", fontsize=26)

axes[0].scatter(X_train[:,0].detach().numpy(), X_train[:,1].detach().numpy())
axes[0].scatter(NoNoise[:,0].detach().numpy(), NoNoise[:,1].detach().numpy())
axes[0].set_xlabel("X1")
axes[0].set_ylabel("X2")
axes[0].set_title("Trained at noise level 0.1", fontsize= 16)
axes[0].legend()
axes[1].scatter(X_Noisy[:,0].detach().numpy(), X_Noisy[:,1].detach().numpy())
axes[1].scatter(Noise[:,0].detach().numpy(), Noise[:,1].detach().numpy(), label= "Noise")
axes[1].set_xlabel("X1")
axes[1].set_ylabel("X2")
axes[1].set_title("Trained at noise level 0.1", fontsize= 16)
axes[1].legend()

axes[2].scatter(X_Noisy[:,0].detach().numpy(), X_Noisy[:,1].detach().numpy())
axes[2].scatter(NoiseT[:,0].detach().numpy(), NoiseT[:,1].detach().numpy(), label= "NoiseT")
axes[2].set_xlabel("X1")
axes[2].set_ylabel("X2")
axes[2].set_title("Trained at noise level 0.2", fontsize= 16)
axes[2].legend()

plt.tight_layout()
plt.show()

```



The reconstructed data resembles the center line of the noisy data much more, when the model is trained at the according noise level.

2. Two-dimensional data without bottleneck

We remove the bottleneck which would usually force the network to focus on the important features. To counteract this, we introduce an additional loss term. We use MMD between the code distribution (push-forward through the encoder) and a two-dimensional standard normal.

The kernel for the MMD shall be a sum of squared exponentials or inverse multiquadratics at multiple bandwidths (hyperparameter). We use the Autoencoder from task 1 with `bottleneck_size = 2`.

```
In [ ]: # Train Autoencoder with bottleneck_size = 2 using MMD loss
model_mmd = Autoencoder(input_size = 2,
                        bottleneck_size = 2,
                        hidden_size = 16,
                        layers = 3)
model_mmd.train_mmd_autoencoder(X_train,
                                training_set_size = 2000,
                                mini_batch_size = 128,
                                n_epochs = 2000,
                                learning_rate = 0.005,
                                mmd_kernel = "squared_exponential",
                                mmd_bandwidths = [0.05, 0.1, 0.2, 0.4, 0.8,
                                lambda_mmd = 1)
```

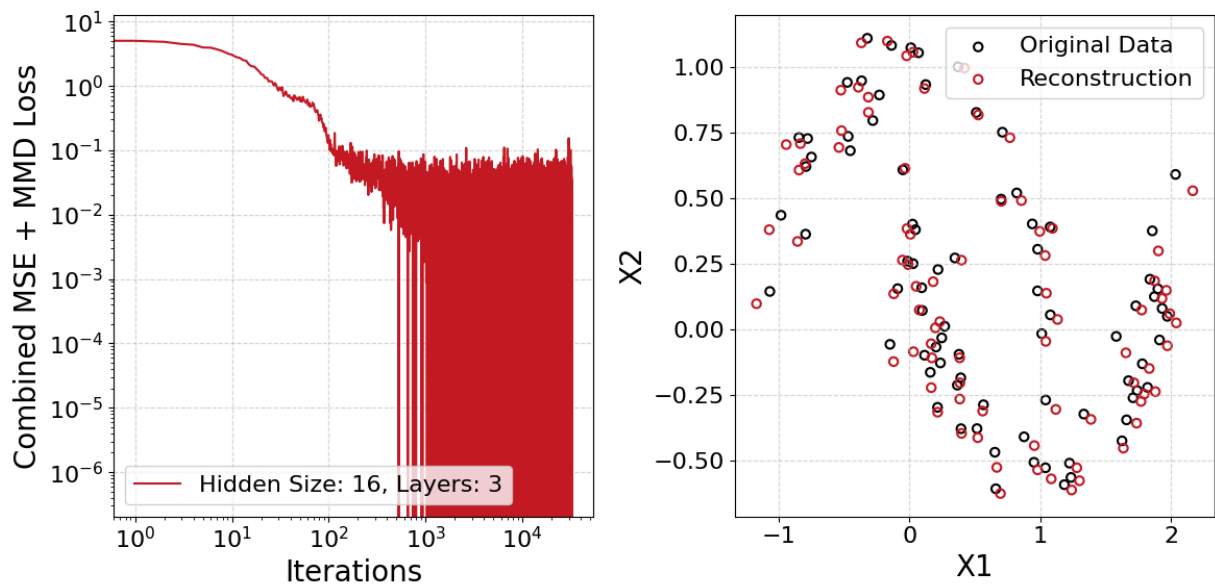
100%|██████████| 2000/2000 [07:46<00:00, 4.29it/s]

```
In [ ]: params = (2000, 128, 2000, 0.005, 16, 3)
results_mmd_losses = []
results_mmd_losses.append((params, model_mmd.train_losses, model_mmd.outputs))
# Plot loss for MMD Autoencoder
fig, axes = plt.subplots(1, 2)
fig.suptitle("Autoencoder Training: \nbottleneck_size = 2, MMD loss", fontsize=12)
# Plot training losses
for res in results_mmd_losses:
    params, train_losses, _ = res
    axes[0].plot(train_losses, label=f"Hidden Size: {params[4]}, Layers: {params[5]}")
axes[0].set_xlabel("Iterations")
axes[0].set_ylabel("Combined MSE + MMD Loss")
axes[0].loglog()
axes[0].legend()

# Plot final reconstruction
# Get final output
final_epoch, X_batch, decoded = results_mmd_losses[-1][2]
print(len(results_mmd_losses))
axes[1].scatter(X_batch[:,0].detach().numpy(), X_batch[:,1].detach().numpy(),
                label="Original Data",
                color = "black")
for i in range(len(results_mmd_losses)):
    final_epoch, X_batch, decoded = results_mmd_losses[i][2]
    axes[1].scatter(decoded[:,0].detach().numpy(), decoded[:,1].detach().numpy(),
                    label=f"Reconstruction",
                    alpha=1,
                    color = colors[i])
axes[1].set_xlabel("X1")
axes[1].set_ylabel("X2")
axes[1].legend()
plt.tight_layout()
plt.show()
```

1

Autoencoder Training: bottleneck_size = 2, MMD loss



As predicted in the exercise description, the reconstruction error is much less ($\approx 10^{-4}$) than in exercise 1 with a `bottleneck_size = 2`. Additionally, plotting the batch data with the corresponding reconstructed points shows how the data has been almost perfectly reconstructed. This learning process is also extremely quick, almost perfectly reconstructing the sample data after less than 20 iterations. Further refinement does take a couple of hundred - thousand epochs of training.

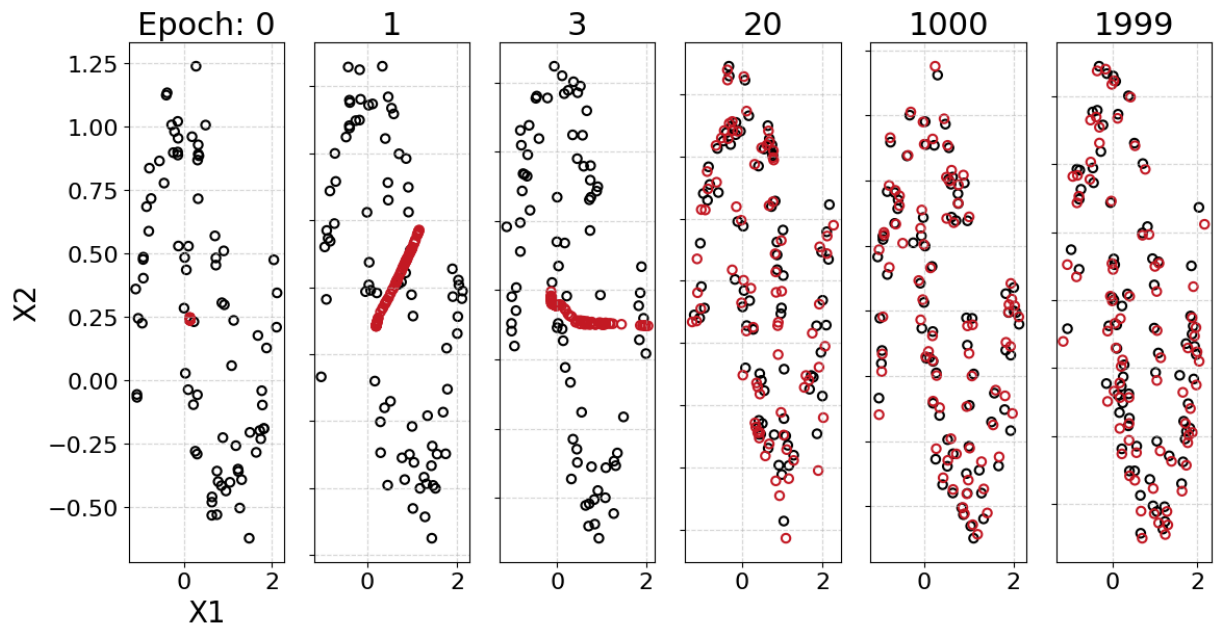
One can also remark that in comparison to `bottleneck_size = 1`, which only provided a 1-dimensional curve, the new distribution is 2-dimensional.

```
In [ ]: fig, axes = plt.subplots(1, 6)
fig.suptitle("MMD Autoencoder Training: Evolution of Reconstruction", fontsize=14)
indices = [-2000, -1999, -1997, -1980, -1000, -1]
for i, idx in enumerate(indices):
    final_epoch, X_batch, decoded = model_mmd.outputs[idx]
    axes[i].scatter(X_batch[:,0].detach().numpy(), X_batch[:,1].detach().numpy(),
                    label="Original Data",
                    color = "black")
    axes[i].scatter(decoded[:,0].detach().numpy(), decoded[:,1].detach().numpy(),
                    label=f"(Epoch: {final_epoch})",
                    alpha=1,
                    color = colors[0])

    axes[i].tick_params(labelleft=False)
    if i == 0:
        axes[i].set_xlabel("X1")
        axes[i].set_ylabel("X2")
        axes[i].tick_params(labelleft=True)
        axes[i].set_title(f"Epoch: {final_epoch}")
    else:
```

```
axes[i].set_title(f"{final_epoch}")
plt.tight_layout()
plt.show()
```

MMD Autoencoder Training: Evolution of Reconstruction



We now want to corroborate that the code distribution after the encoder resembles a standard Gaussian. Superimposing the results over samples from a standard Gaussian shows great results. Calculating the mean and covariance of the code distribution also yields

```
In [ ]: # Generate samples from 2d standard normal
normal_samples = torch.randn((1000, 2))

# Plot codes after training MMD Autoencoder
with torch.no_grad():
    codes = X_test[:1000]
    for layer in model_mmd.encoder:
        codes = layer(codes)

    # Calculate mean and covariance of codes
    mu = torch.mean(codes, dim=0)
    cov = torch.cov(codes.T)

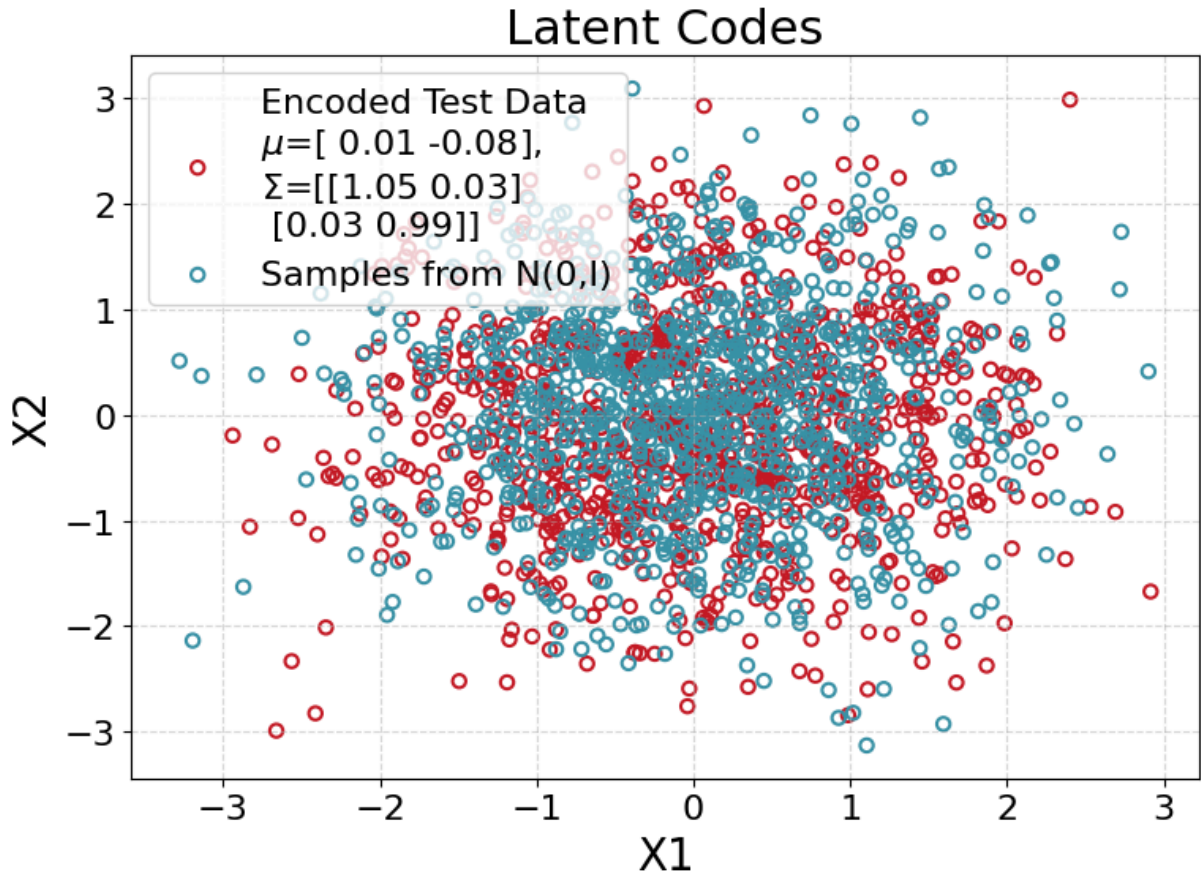
# Calculate t-test to see if codes and samples are from same distribution
from scipy.stats import ttest_ind
t_statistic_x1, p_value_x1 = ttest_ind(codes[:, 0], normal_samples[:, 0])
t_statistic_x2, p_value_x2 = ttest_ind(codes[:, 1], normal_samples[:, 1])
print(f"T-test results for X1: t-statistic = {t_statistic_x1}, p-value = {p_value_x1}")
print(f"T-test results for X2: t-statistic = {t_statistic_x2}, p-value = {p_value_x2}")

fig = plt.figure(figsize=(8, 6))
plt.scatter(codes[:, 0].detach().numpy(), codes[:, 1].detach().numpy(), alpha=0.5,
            label = f"Encoded Test Data\n $\mu = \{mu.detach().numpy().round(2)\}$ ")
plt.scatter(normal_samples[:, 0].detach().numpy(), normal_samples[:, 1].detach().numpy(), alpha=0.5,
            label = f"Normal Samples\n $\mu = \{mu.detach().numpy().round(2)\}$ ")
```

```
plt.title("Latent Codes")
plt.legend()
plt.xlabel("X1")
plt.ylabel("X2")
plt.tight_layout()
plt.show()
```

T-test results for X1: t-statistic = 0.024092652964352607, p-value = 0.9807811095550562

T-test results for X2: t-statistic = -3.2402617193968934, p-value = 0.0012138213905118105



Next we investigate briefly the influence of the weight of the MMD^2 Loss term. Starting with $\lambda_{MMD} = 1$ to 1000, training for 1000 epochs each, we see the

```
In [ ]: # 1 Investigate the influence of lambda weights fixed other hyperparameters
MMD_weights = [1, 5, 10, 20, 50, 100, 500, 1000]

results_MMD_weights = []
params = (2000, 128, 1000, 0.005, 16, 3)
for weight in MMD_weights:
    print(f"Training with MMD weight: {weight}")
    # Create model
    model_MMD_w = Autoencoder(input_size=2,
                              bottleneck_size=2,
                              hidden_size=params[4],
                              layers=params[5])

    # Train model
    model_MMD_w.train_mmd_autoencoder(X_train,
```



```

        training_set_size=params[0],
        mini_batch_size=params[1],
        n_epochs=params[2],
        learning_rate=params[3],
        mmd_kernel = "inverse_multi_quadratic",
        mmd_bandwidths = [0.05, 0.1, 0.2, 0.4, 0.8, 1.6,
                           lambda_mmd = weight)

# Save result
results_MMD_weights.append((params, model_MMD_w.train_losses, model_MMD_

```

Training with MMD weight: 1

100%|██████████| 1000/1000 [06:45<00:00, 2.47it/s]

Training with MMD weight: 5

100%|██████████| 1000/1000 [03:35<00:00, 4.65it/s]

Training with MMD weight: 10

100%|██████████| 1000/1000 [03:47<00:00, 4.39it/s]

Training with MMD weight: 20

100%|██████████| 1000/1000 [03:53<00:00, 4.28it/s]

Training with MMD weight: 50

100%|██████████| 1000/1000 [03:56<00:00, 4.22it/s]

Training with MMD weight: 100

100%|██████████| 1000/1000 [03:56<00:00, 4.23it/s]

Training with MMD weight: 500

100%|██████████| 1000/1000 [03:56<00:00, 4.23it/s]

Training with MMD weight: 1000

100%|██████████| 1000/1000 [04:02<00:00, 4.13it/s]

```

In [ ]: # Plot code distribution for different MMD weights
fig, axes = plt.subplots(2, 4)
fig.suptitle("MMD Autoencoder Training: Influence of MMD Weight on Code Dist
# Plot code distributions

for i in range(len(results_MMD_weights)):
    with torch.no_grad():
        codes = X_test[:1000]
        model = results_MMD_weights[i][2]
        for layer in model.encoder:
            codes = layer(codes)

# Calculate mean and covariance of codes
mu = torch.mean(codes, dim=0)
cov = torch.cov(codes.T)

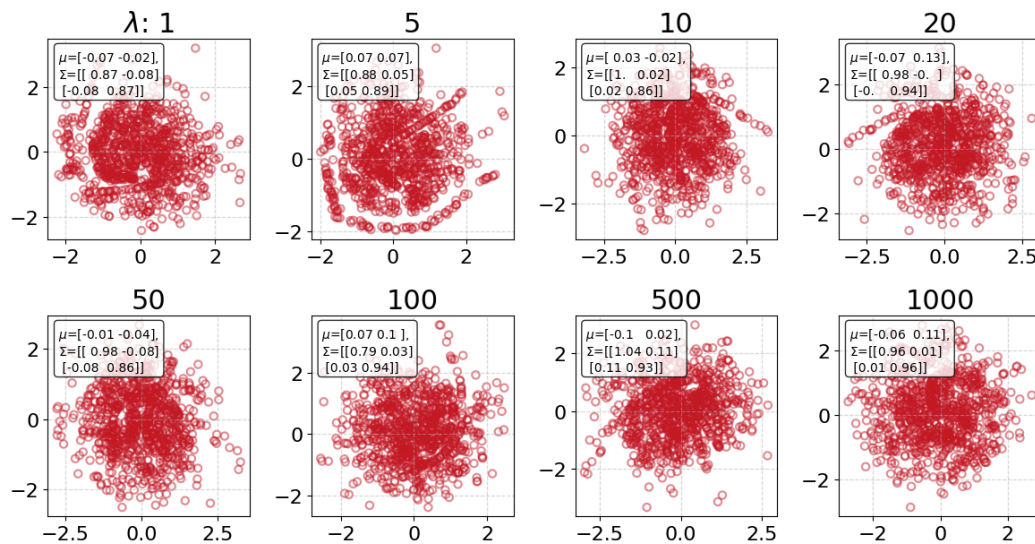
axes[i // 4, i % 4].scatter(codes[:, 0].detach().numpy(), codes[:, 1].de
                           alpha = 0.5)
label = f"$\mu$={mu.detach().numpy().round(2)}, \n$\Sigma$={cov.detach()
axes[i // 4, i % 4].text(0.05, 0.95, label, transform=axes[i // 4, i % 4
                           fontsize=10, verticalalignment='top', bbox=textbox_style)

if i == 0:
    axes[i // 4, i % 4].set_title(f"$\lambda$: {MMD_weights[i]}")
else:
    axes[i // 4, i % 4].set_title(f"{MMD_weights[i]}")

```

```
plt.tight_layout()
plt.show()
```

MMD Autoencoder Training: Influence of MMD Weight on Code Distribution



As we see, for larger weights of the MMD² loss, the distribution approaches more closely a standard normal. However, as we can also see in the next plots, the reconstruction becomes worse as well. There is a balance which must be kept between reconstruction and MMD² error.

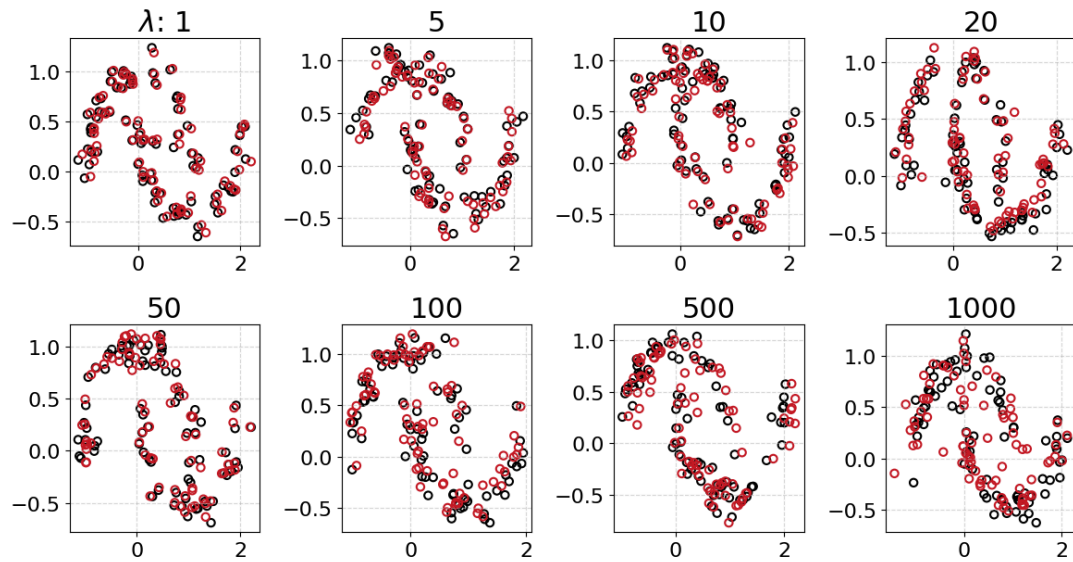
```
In [ ]: # Plot reconstruction for different MMD weights
fig, axes = plt.subplots(2, 4)
fig.suptitle("MMD Autoencoder Training: Influence of MMD Weight on Reconstru")
# Plot code distributions

for i in range(len(results_MMD_weights)):
    with torch.no_grad():
        final_epoch, X_batch, decoded = results_MMD_weights[i][2].outputs[-1]
        axes[i // 4, i % 4].scatter(X_batch[:,0].detach().numpy(), X_batch[:,1].detach().numpy(),
                                     label="Original Data",
                                     color = "black")
        axes[i // 4, i % 4].scatter(decoded[:,0].detach().numpy(), decoded[:,1].detach().numpy(),
                                     label=f"(Epoch: {final_epoch})",
                                     alpha=1,
                                     color = colors[0])

    #axes[i // 4, i % 4].tick_params(labelleft=False)
    if i == 0:
        #axes[i // 4, i % 4].set_xlabel("X1")
        #axes[i // 4, i % 4].set_ylabel("X2")
        #axes[i // 4, i % 4].tick_params(labelleft=True)
        axes[i // 4, i % 4].set_title(f"$\lambda$: {MMD_weights[i]}")
    else:
        axes[i // 4, i % 4].set_title(f"{MMD_weights[i]}")

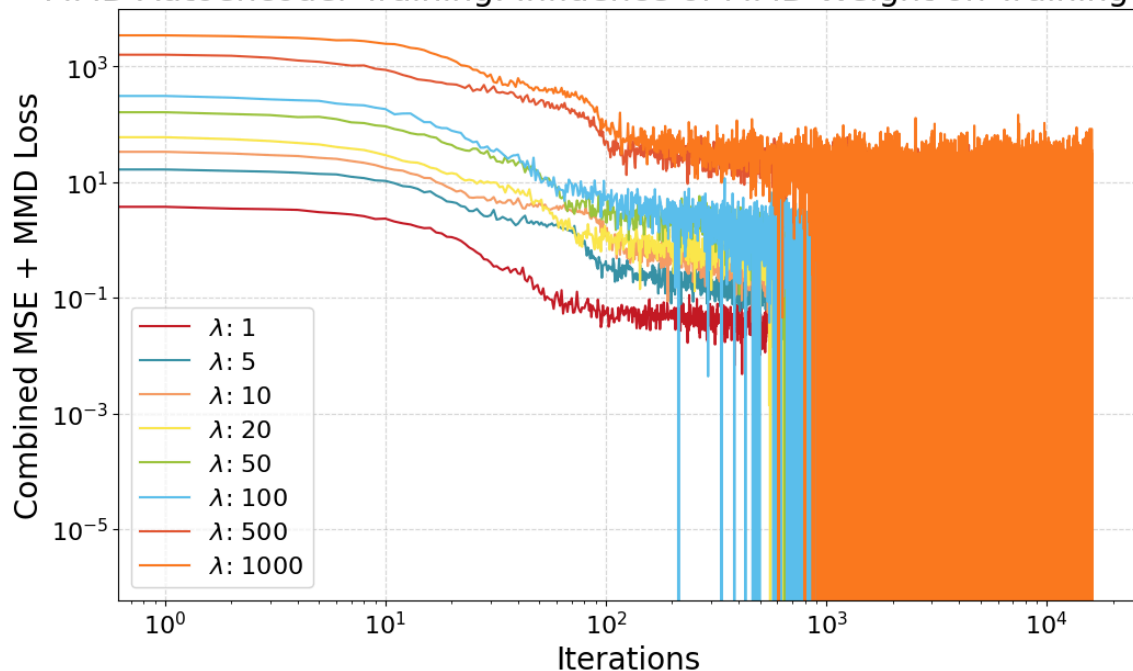
plt.tight_layout()
plt.show()
```


MMD Autoencoder Training: Influence of MMD Weight on Reconstruction



```
In [ ]: # Plot the training losses for different MMD weights
plt.figure()
for res in results_MMD_weights:
    params, train_losses, _ = res
    plt.plot(train_losses, label=f"$\lambda$: {MMD_weights[results_MMD_weight]}")
plt.xlabel("Iterations")
plt.ylabel("Combined MSE + MMD Loss")
plt.title("MMD Autoencoder Training: Influence of MMD Weight on Training Loss")
plt.loglog()
plt.legend()
plt.show()
```

MMD Autoencoder Training: Influence of MMD Weight on Training Loss



3. Higher-dimensional data

```
In [ ]: # Test AE code with bottleneck sizes 2, 4, 8 on MNIST dataset
from sklearn.datasets import load_digits
digits = load_digits()
X_mnist = digits.data / 16.0 # Normalize pixel values to [0, 1]
y_mnist = digits.target
X_mnist = torch.tensor(X_mnist, dtype=torch.float32)
y_mnist = torch.tensor(y_mnist, dtype=torch.long)
from sklearn.model_selection import train_test_split
X_mnist_train, X_mnist_test, y_mnist_train, y_mnist_test = train_test_split(
```

In Ex. 1 we saw that the hyperparameters that have the biggest influence are network size and depth, mini-batch size and training time/epochs. So we will only investigate these.

```
In [ ]: # Show sample of mnist digit reconstruction for bottleneck_sizes 2,4,8
bottleneck_sizes = [2, 4, 8]
reconstruction_results = []
params = [10000, 256, 200, 0.001, 32, 2] # training_set_size, mini_batch_size, n_epochs, mini_batch_size, learning_rate, layers

for size in bottleneck_sizes:
    print(f"Training with bottleneck size: {size}")

    model_size = Autoencoder(
        input_size=64,
        bottleneck_size=size,
        hidden_size=params[4],
        layers=params[5]
    )

    model_size.train_autoencoder(X_mnist_train,
                                training_set_size=params[0],
                                n_epochs=params[2],
                                mini_batch_size=params[1],
                                learning_rate=params[3]
    )

    reconstruction_results.append((size, model_size.train_losses, model_size.train_losses[-1]))
    print(f"Loss: {model_size.train_losses[-1]}, for bottleneck size: {size}")
```

Training with bottleneck size: 2

```
100%|██████████| 200/200 [00:04<00:00, 42.94it/s]
Loss: 0.03675268217921257, for bottleneck size: 2
Training with bottleneck size: 4
```

```
100%|██████████| 200/200 [00:03<00:00, 53.73it/s]
Loss: 0.02747570350766182, for bottleneck size: 4
Training with bottleneck size: 8
```

```
100%|██████████| 200/200 [00:03<00:00, 52.22it/s]
Loss: 0.020401477813720703, for bottleneck size: 8
```

```
In [ ]: # Show reconstruction results for bottleneck_sizes 2,4,8
for size, losses, model in reconstruction_results:
    # Get some test samples and their reconstructions
    with torch.no_grad():
```

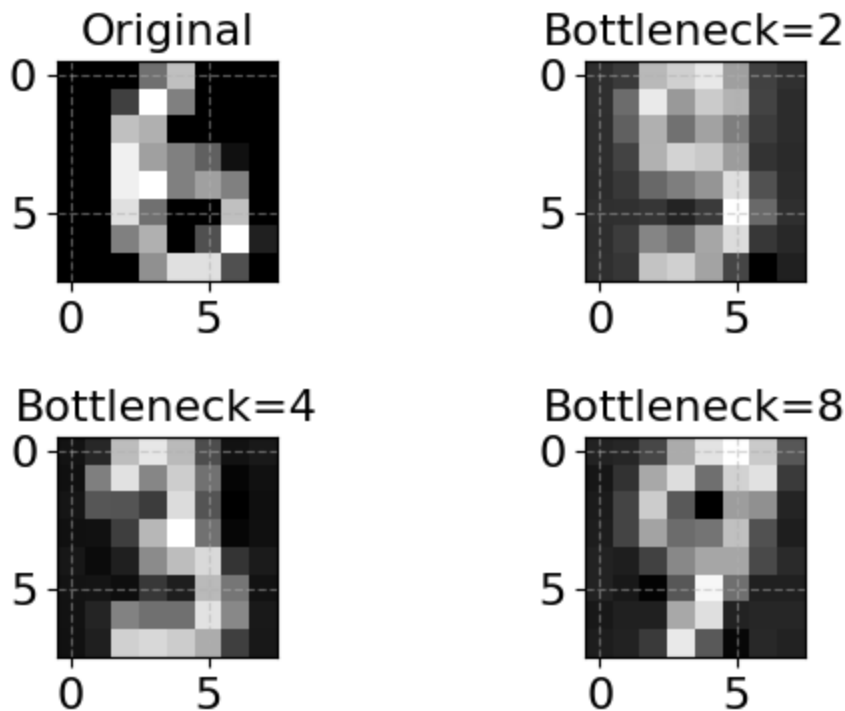
```

test_samples = X_mnist_test[:4]
reconstructed_samples = model.forward(test_samples)

fig, axes = plt.subplots(2, 2, figsize=(6, 4))
for i in range(4):
    if i == 0:
        axes[i//2, i%2].set_title("Original", fontsize=16)
        axes[i//2, i%2].imshow(test_samples[i].reshape(8, 8), cmap='gray')
    else:
        axes[i//2, i%2].imshow(reconstructed_samples[i].reshape(8, 8), cmap='gray')
        axes[i//2, i%2].set_title(f"Bottleneck={bottleneck_sizes[i - 1]}", fontsize=16)

plt.tight_layout()
plt.show()

```

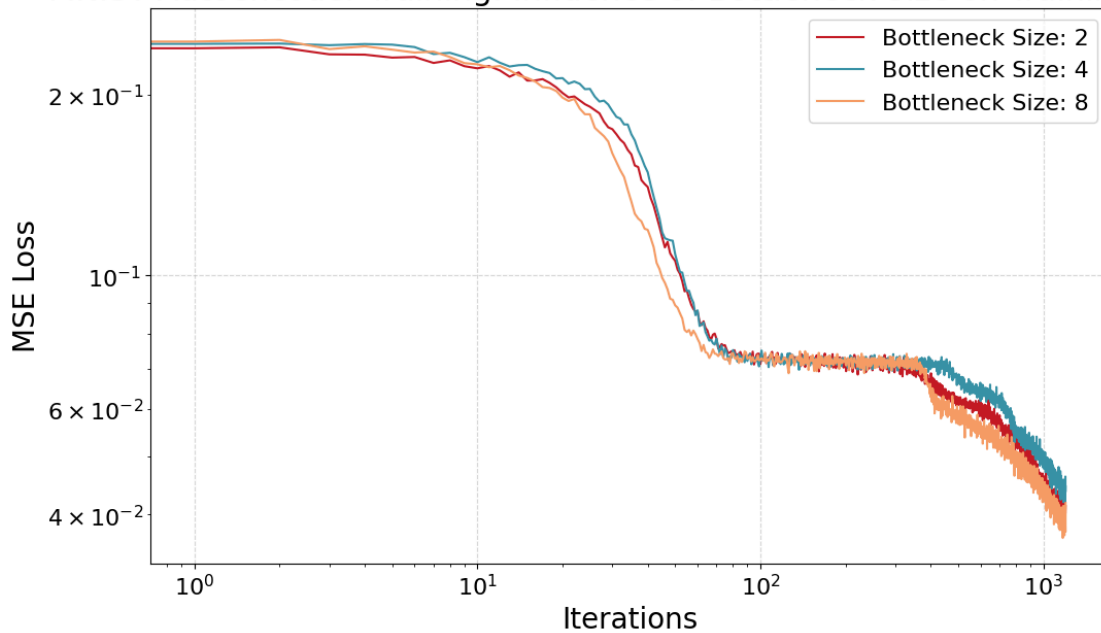


```

In [ ]: # Plot MSE loss for MMD Autoencoder
plt.figure()
for res in reconstruction_results:
    size, train_losses, _ = res
    plt.plot(train_losses, label=f"Bottleneck Size: {size}")
plt.xlabel("Iterations")
plt.ylabel("MSE Loss")
plt.title("MNIST Autoencoder Training: Influence of Bottleneck Size on Train")
plt.loglog()
plt.legend()
plt.show()

```

MNIST Autoencoder Training: Influence of Bottleneck Size on Training Loss



```
In [ ]: # layers Show sample of mnist digit reconstruction for bottleneck_sizes 2,4,
bottleneck_sizes = [2, 4, 8]
reconstruction_results = []
params = [10000, 256, 200, 0.001, 32, 4] # training_set_size, mini_batch_size, n_epochs, learning_rate, hidden_size, layers

for size in bottleneck_sizes:
    print(f"Training with bottleneck size: {size}")

    model_size = Autoencoder(
        input_size=64,
        bottleneck_size=size,
        hidden_size=params[4],
        layers=params[5]
    )

    model_size.train_autoencoder(X_mnist_train,
                                training_set_size=params[0],
                                n_epochs=params[2],
                                mini_batch_size=params[1],
                                learning_rate=params[3]
    )

    reconstruction_results.append((size, model_size.train_losses, model_size
```

Training with bottleneck size: 2

100%|██████████| 200/200 [00:05<00:00, 37.59it/s]

Training with bottleneck size: 4

100%|██████████| 200/200 [00:05<00:00, 39.61it/s]

Training with bottleneck size: 8

100%|██████████| 200/200 [00:04<00:00, 43.44it/s]

```
In [ ]: # Show reconstruction results for bottleneck_sizes 2,4,8
for size, losses, model in reconstruction_results:
    # Get some test samples and their reconstructions
```

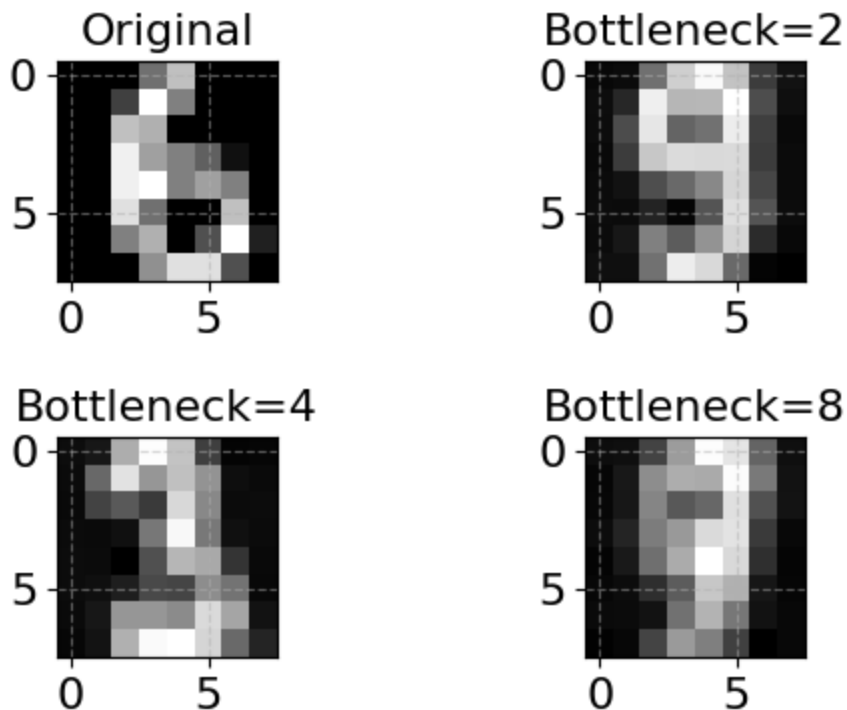
```

with torch.no_grad():
    test_samples = X_mnist_test[:4]
    reconstructed_samples = model.forward(test_samples)

fig, axes = plt.subplots(2, 2, figsize=(6, 4))
for i in range(4):
    if i == 0:
        axes[i//2, i%2].set_title("Original", fontsize=16)
        axes[i//2, i%2].imshow(test_samples[i].reshape(8, 8), cmap='gray')
    else:
        axes[i//2, i%2].imshow(reconstructed_samples[i].reshape(8, 8), cmap='gray')
        axes[i//2, i%2].set_title(f"Bottleneck={bottleneck_sizes[i - 1]}", f

plt.tight_layout()
plt.show()

```

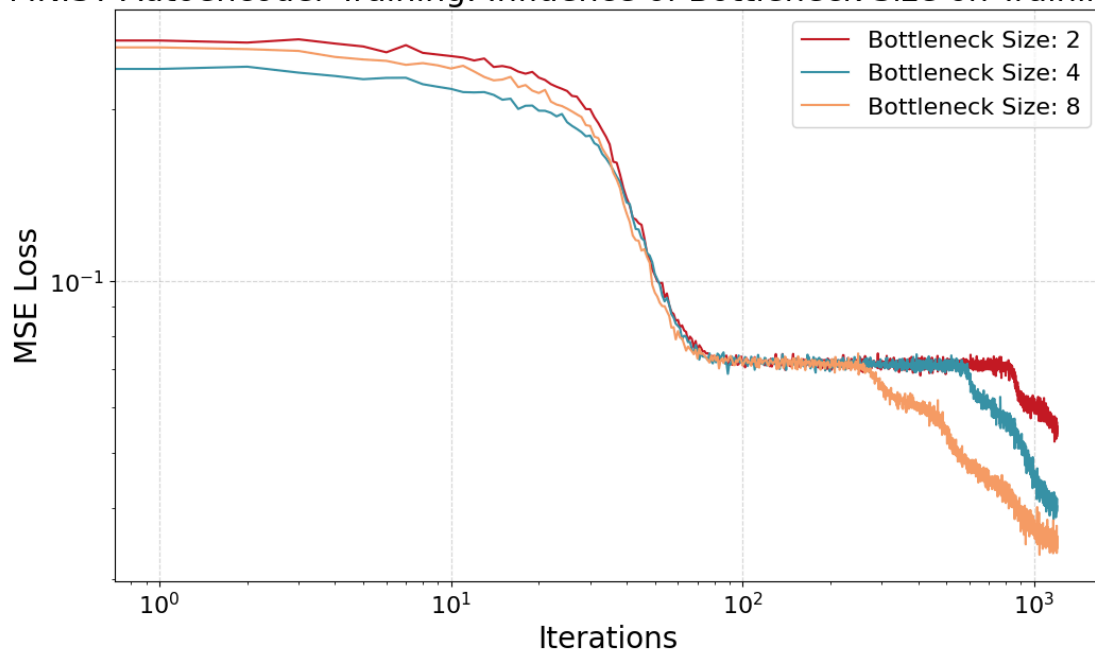


```

In [ ]: # Plot MSE loss for MMD Autoencoder
plt.figure()
for res in reconstruction_results:
    size, train_losses, _ = res
    plt.plot(train_losses, label=f"Bottleneck Size: {size}")
plt.xlabel("Iterations")
plt.ylabel("MSE Loss")
plt.title("MNIST Autoencoder Training: Influence of Bottleneck Size on Train")
plt.loglog()
plt.legend()
plt.show()

```

MNIST Autoencoder Training: Influence of Bottleneck Size on Training Loss



We settle on a specific architecture and move on from there

```
In [ ]: bottleneck_sizes = [2, 4, 8]
results_mnist = []
params = [5000, 128, 500, 0.001, 256, 2]

for size in bottleneck_sizes:
    model_mnist = Autoencoder(input_size=64,
                              bottleneck_size=size,
                              hidden_size=params[4],
                              layers=params[5])
    model_mnist.train_autoencoder(X_mnist_train,
                                  training_set_size=params[0],
                                  mini_batch_size=params[1],
                                  n_epochs=params[2],
                                  learning_rate=params[3])
    results_mnist.append((size, model_mnist.train_losses, model_mnist))
```

```
100%|██████████| 500/500 [00:34<00:00, 14.63it/s]
100%|██████████| 500/500 [00:42<00:00, 11.83it/s]
100%|██████████| 500/500 [00:45<00:00, 11.10it/s]
```

Now we train a Random Forest Classifier and check if the predicted labels from our different models

```
In [ ]: # Train Random Forest Classifier on data, use it to classify reconstructions
# Save weights of the classifier for later use
from sklearn.ensemble import RandomForestClassifier
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X_mnist_train.numpy(), y_mnist_train)
```

▼ RandomForestClassifier ⓘ ?

► Parameters

```
In [ ]: # Check reconstruction for different bottleneck sizes
fig, axes = plt.subplots(1, len(bottleneck_sizes) + 1, figsize=(12, 4))
fig.suptitle("MNIST Autoencoder: Influence of Bottleneck Size on Reconstruct
# Original image
original_image = X_mnist_test[0].reshape(8, 8)
axes[0].imshow(original_image.detach().numpy(), cmap='gray')
axes[0].set_title("Original Image")
axes[0].axis('off')

# Reconstructed images
for i in range(len(results_mnist)):
    size, _, model = results_mnist[i]
    with torch.no_grad():
        code = X_mnist_test[torch.randint(0, X_mnist_test.size(0), (1,))].ur
        decoded = model.forward(code)

        reconstructed_image = decoded.reshape(8, 8)
        # Predict label with RF classifier
        #print(reconstructed_image.shape)
        #print(decoded.detach().numpy())
        predicted_label = rf_classifier.predict(decoded.numpy().reshape(1, -1))
        #print(f"Bottleneck: {size}, \nLabel: {predicted_label[0]}")
        axes[i + 1].imshow(reconstructed_image.detach().numpy(), cmap='gray')
        axes[i + 1].set_title(f"Bottleneck Size: {size}, \nLabel: {predicted_label[0]}")
        axes[i + 1].axis('off')
plt.tight_layout()
plt.show()
```

MNIST Autoencoder: Influence of Bottleneck Size on Reconstruction

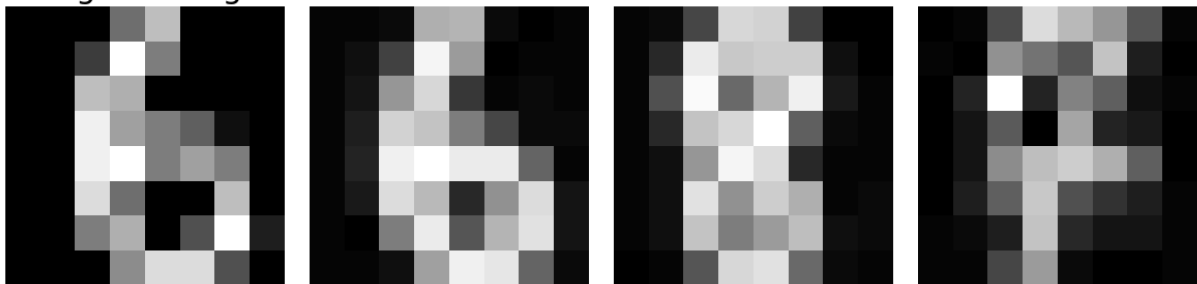
Bottleneck Size: 2, Bottleneck Size: 4, Bottleneck Size: 8,

Original Image

Label: 6

Label: 8

Label: 7

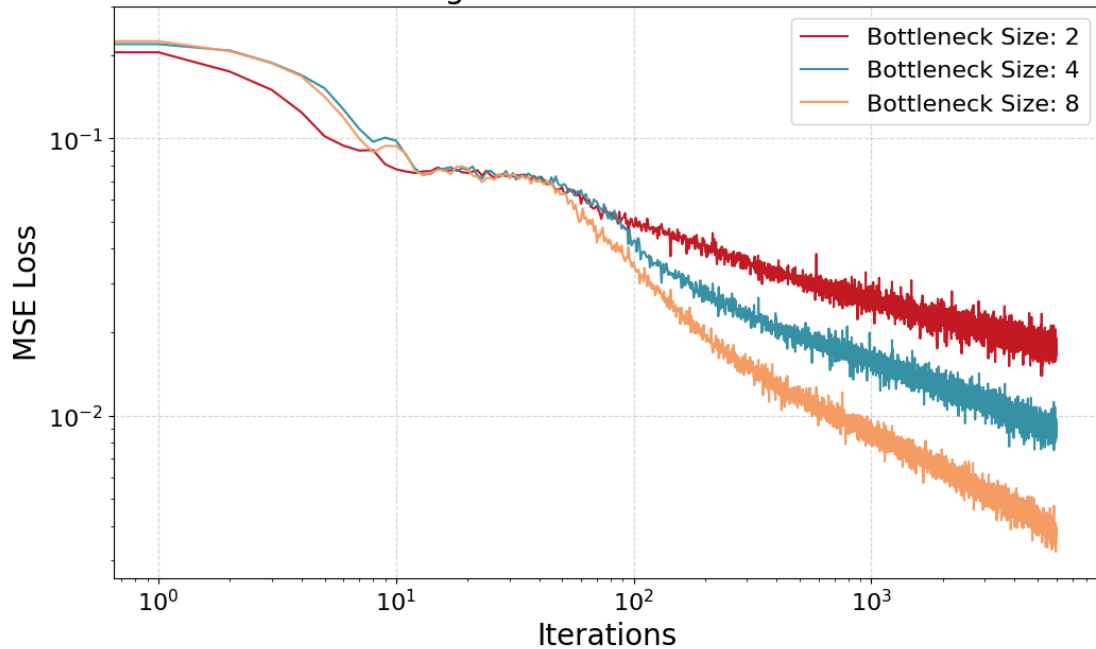


```
In [ ]: # Plot MSE loss for MMD Autoencoder
plt.figure()
for res in results_mnist:
    size, train_losses, _ = res
    plt.plot(train_losses, label=f"Bottleneck Size: {size}")
plt.xlabel("Iterations")
plt.ylabel("MSE Loss")
plt.title("MNIST Autoencoder Training: Influence of Bottleneck Size on Train
plt.loglog()
```



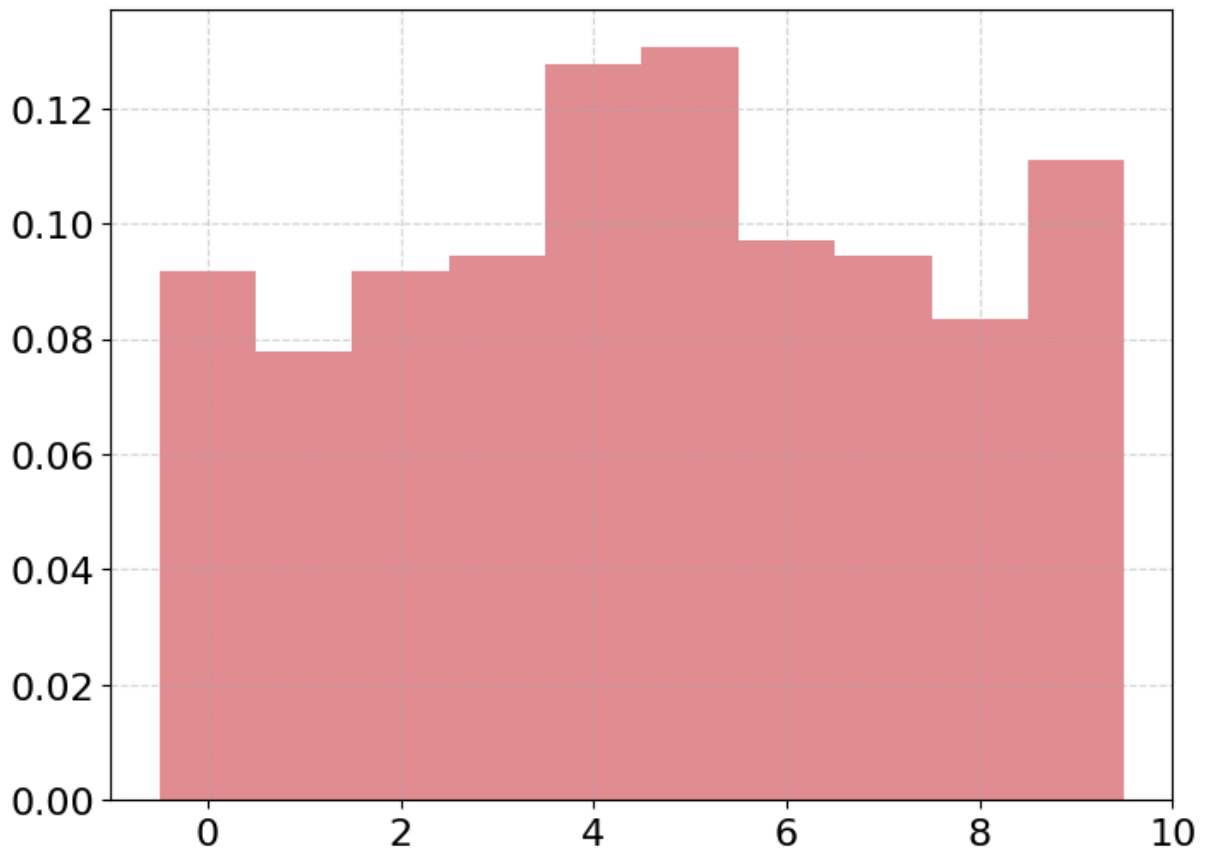
```
plt.legend()
plt.show()
```

MNIST Autoencoder Training: Influence of Bottleneck Size on Training Loss



```
In [ ]: # Classify the reconstructions
predictions_mnist = []
for res in results_mnist:
    size, _, model = res
    with torch.no_grad():
        decoded = model.forward(X_mnist_test)
        preds = rf_classifier.predict(decoded.numpy())
        predictions_mnist.append((size, preds))

# Plot histogram prediction
plt.figure(figsize = (8,6))
plt.hist(y_mnist_test.numpy(), torch.arange(11)-0.5, alpha=0.5, label="True")
plt.show()
```

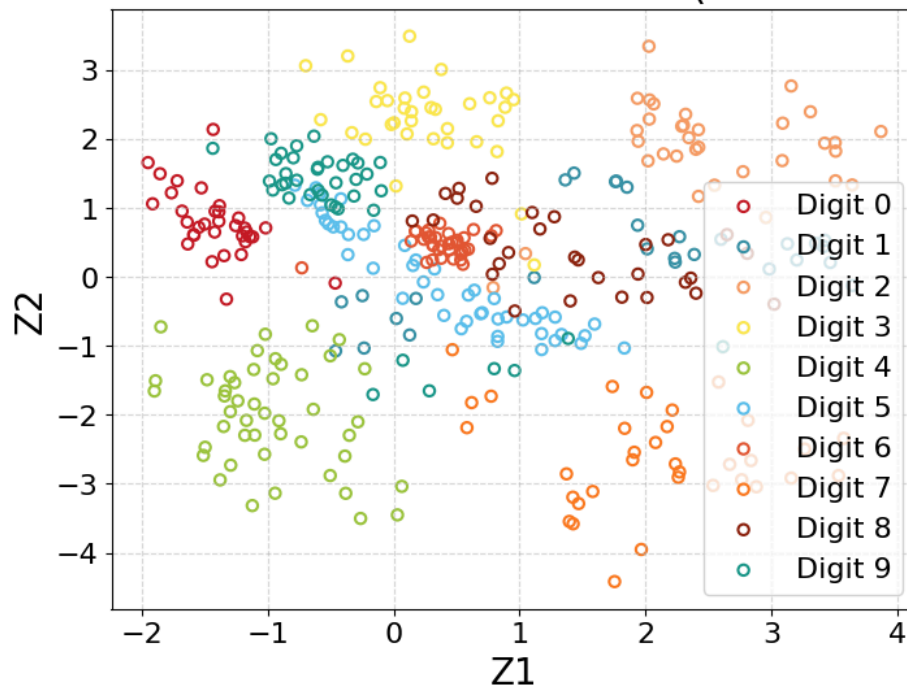


```
In [ ]: # Visualize code distribution for bottleneck_size = 2
with torch.no_grad():
    codes = X_mnist_test
    for layer in results_mnist[0][2].encoder:
        codes = layer(codes)

codes_np = codes.numpy()
plt.figure(figsize=(8,6))
for digit in range(10):
    mask = (y_mnist_test.numpy() == digit)
    plt.scatter(codes_np[mask, 0], codes_np[mask, 1], label=f"Digit {digit}")
#plt.scatter(codes_np[:, 0], codes_np[:, 1], alpha=0.7)
#plt.colorbar(ticks=range(10), label='Digit Label')
#plt.clim(-0.5, 9.5)
plt.legend()
plt.title("MNIST Autoencoder: Code Distribution (Bottleneck Size = 2)")
plt.xlabel("Z1")
plt.ylabel("Z2")
```

Text(0, 0.5, 'Z2')

MNIST Autoencoder: Code Distribution (Bottleneck Size = 2)



One can see that the codes for the different digits are clustered together.

In []: